MODULE 5

深度學習：從神經網路到 CNN

M5 — Deep Learning: Architectures & Practice

兩週弄清楚三件事：神經網路到底在算什麼、 Layers 怎麼設計、
以及最重要的——什麼時候「不該」用深度學習。

資料科學　2027 Spring　·　第 8–9 週　·　Phase B　｜　謝楠楨

本講義大綱（W8–W9 共兩週）

01 誰決定特徵 · 傳統 ML 與深度學習最核心的差別

02 從邏輯斯迴歸出發 · 你已經會一半了

03 前向傳播手算 · 一個網路實際在算什麼

04 激活函數 · 沒有它，再深也等於一條直線

05 梯度下降與反向傳播 · 訓練在做什麼

06 Layer 堆疊 · Dense · ReLU · Dropout · 輸出層

07 輸出層設定 · 最容易設錯的一層

08 標準化與權重初始化 · 做錯會整個訓練不起來

09 表格資料的嵌入層 · 深度學習少數的真正優勢

10 Epoch / Batch / Iteration · 三個最常搞混的詞

11 讀懂 Loss 曲線 · 四種樣態與處方

12 對付過擬合的六招 · 動手順序很重要

13 AI Studio 的三種實作 · Neural Net · H2O · DL4J

14 卷積運算手算 · 一步一步算給你看

15 池化與 CNN 流程 · 層級特徵是怎麼長出來的

16 ★ 經典架構與設計理由 · SimpleCNN · VGG · ResNet · Xception

17 架構設計思想的演進 · 每一代解決了什麼

18 遷移學習三種策略 · 不要從零開始訓練

19 ★ 什麼時候不該用 · 本模組最重要的一節

20 醫學影像的六個挑戰 · 含捷徑學習的真實案例

21 自動化與超參數最佳化 · W9 實作

22 四大架構家族 · MLP · CNN · RNN · Transformer

23 模型的一生 · 訓練、部署、監測、再訓練

24 十個常見錯誤 · 依發生頻率排序

25 Take-Home Assignment · 模型比較表

兩週的安排

週次	主題	內容	實作
W8	深度學習 I： 基礎與架構	神經網路運作原理、 Layers 設計、訓練過程、過擬合對策	Iris → Titanic： 逐步加深網路
W9	深度學習 II： CNN 與實務	卷積與池化、 CNN 架構、遷移學習、何時不該用、醫療影像考量	MNIST 手寫數字辨識、 Fine-tune 、自動最佳化

★ W8 建立直覺， W9 處理實務。作業二在 W8 發布、 W12 繳交。

PART 01

深度學習到底是什麼

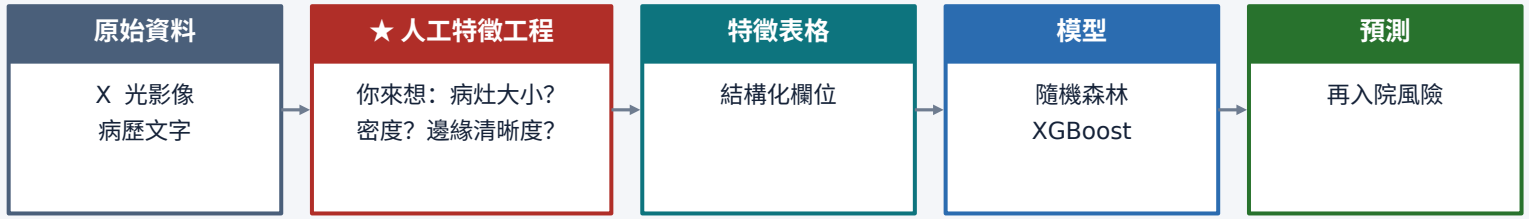
What Deep Learning Actually Is

它不是「更厲害的機器學習」。

它與 M1–M4 的差別只有一個，但那個差別改變了一切。

傳統機器學習 vs 深度學習：誰決定特徵

傳統機器學習（M1–M4）：特徵由「你」決定



深度學習（M5）：特徵由「模型」自己學



★ 這是最核心的差別：深度學習不是「更厲害的模型」，是「把特徵工程交給模型」。

傳統機器學習

★ 特徵由「你」決定。你要想出「病灶大小」「密度」
「邊緣清晰度」這些欄位，模型才有東西可學。

深度學習

★ 特徵由「模型」自己學。你直接把原始影像丟進去，
隱藏層會自己學出有用的表徵。

這是最核心的差別

不是「模型比較強」，是「把特徵工程交給模型」。

★ 這既是威力也是代價

威力：處理影像語音時人根本想不出特徵。
代價：你不知道它學到了什麼（黑箱）。

原講義： RapidMiner_Deep_Learning_Architectures_v3.pdf ★原生圖形，可直接在 PowerPoint 編輯

三個必須先建立的觀念

① 深度學習沒有推翻前面四個模組

- M1 的資料清理、 M2 的評估指標、 M4 的過擬合觀念，全部照樣適用
- ★ 甚至更重要：深度學習更容易過擬合，也更難察覺
- 「換成深度學習就不用做資料清理」是完全錯誤的想法

② 「深」指的是層數，不是難度

- 隱藏層超過一層即可稱「深」，沒有嚴格的層數門檻
- ★ 層數不是越多越好——資料量不夠時，深反而比淺差

③ 它是專門工具，不是萬用升級

- ★ 影像、語音、長文本 → 深度學習有壓倒性優勢
- 表格資料（病歷欄位、檢驗值、行政資料）→ 樹模型長期領先
- 而醫護領域的資料，絕大多數是表格資料。這一點本模組會反覆強調

※ 這一頁的第③點是整個模組的主軸。如果你只記得一件事，請記得它。

PART 02

神經網路實際在算什麼

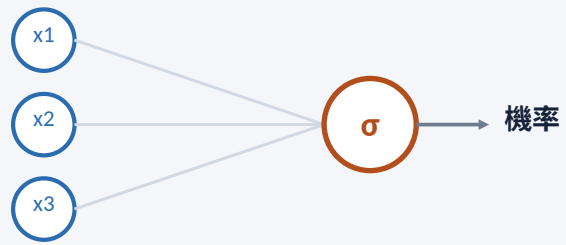
How a Neural Network Computes

拆開來看，裡面沒有魔法。

只有加權求和、非線性轉換，然後重複幾百萬次。

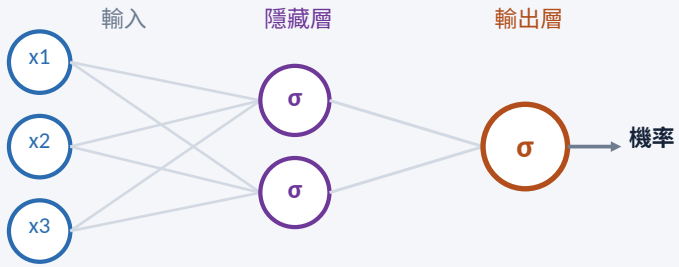
從邏輯斯迴歸到神經網路：其實你已經會一半了

邏輯斯迴歸（M2-2 學過）



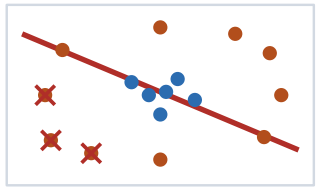
$z = w1 \cdot x1 + w2 \cdot x2 + w3 \cdot x3 + b \quad \rightarrow \quad \sigma(z)$

神經網路（本模組）



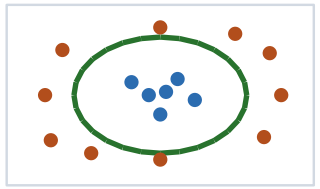
多個邏輯斯迴歸並排，再接一個邏輯斯迴歸

邏輯斯迴歸：只能畫一條直線



✗ 怎麼畫都分不開

★ 神經網路：任意複雜的邊界



✓ 圈起來就分開了

★ 一個神經網路，本質上就是「一堆邏輯斯迴歸串起來」

- 單一神經元 = 加權求和 + 激活函數 = 一個邏輯斯迴歸
- 隱藏層 = 好幾個邏輯斯迴歸並排，各自學不同的組合
- 輸出層 = 再用一個邏輯斯迴歸，整合成最終答案

★ 所以 M2-2 學過的東西沒有白學 —— 深度學習是它的延伸

★ 那多出來的隱藏層帶來什麼？答案：能學出「特徵的組合」。

邏輯斯迴歸只能畫一條直線，神經網路能畫任意複雜的邊界。

單一神經元 = 一個邏輯斯迴歸

加權求和 + 激活函數。
若激活用 Sigmoid，那就是 M2-2 學過的東西。

隱藏層 = 好幾個並排

各自學不同的特徵組合。

輸出層 = 再一個邏輯斯迴歸

把隱藏層的結果整合成最終答案。

★ 多出來的隱藏層帶來什麼

能學出「特徵的組合」。邏輯斯迴歸只能畫一條直線，
神經網路能畫任意複雜的邊界。

★ 原生圖形，可直接在 PowerPoint 編輯

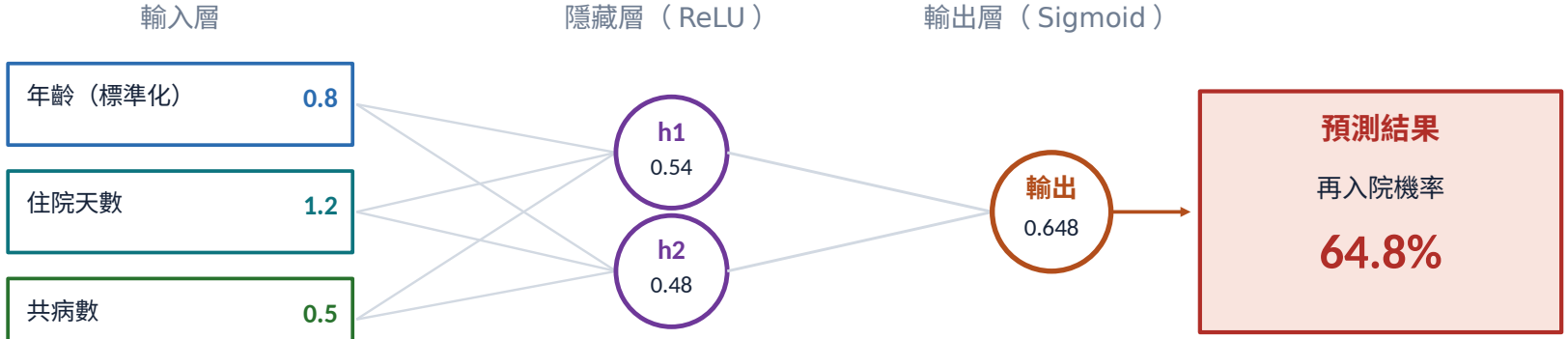
深度學習沿用了哪些前面學過的東西

來自	觀念	在深度學習裡的樣子
M1 資料工程	標準化／正規化	★ 必做。 H2O 預設 standardize=true—— 沒標準化的話梯度會嚴重失衡
M1 資料工程	缺值處理	H2O 預設用平均數填補；但是否恰當要看 MCAR/MAR/MNAR
M2-2 分類	邏輯斯迴歸與 Sigmoid	單一神經元就是它
M2-2 分類	混淆矩陣、 Recall 、 AUC	評估方式完全相同
M2-2 分類	類別不平衡	★ 深度學習一樣會被多數類淹沒； 處理手法也一樣（權重、重抽樣）
M4 集成	偏誤—變異、過擬合	Dropout 與早停就是在對付高變異
M4 集成	交叉驗證	★ 深度學習訓練慢，常改用單一驗證集 + 早停

★ 這張表要傳達的訊息：深度學習不是「另一套系統」，它站在你前七週學的所有東西之上。

前向傳播：完整手算一次

範例：用三個特徵預測 30 天再入院



每一條線都是一個權重 w，訓練就是在調這些數字

完整手算（數值已驗算）

h1 的加權和	z = 0.5×0.8 + (−0.3)×1.2 + 0.8×0.5 + 0.1 = 0.54	**ReLU(0.54) = 0.54**
h2 的加權和	z = 0.2×0.8 + 0.6×1.2 + (−0.4)×0.5 + (−0.2) = 0.48	**ReLU(0.48) = 0.48**
輸出層	z = 1.2×0.54 + (−0.7)×0.48 + 0.3 = 0.612	**Sigmoid(0.612) = 0.648**
損失（若真實 y = 1）	交叉熵 = −ln(0.648)	**= 0.433**

★ 整個神經網路就是這樣一層層算下去。複雜的不是單一步驟，是「有幾百萬個權重要調」。

每個神經元做兩件事

① 加權求和（輸入 × 權重 + 偏差）
② 通過激活函數。就這樣。

實際數值

h1 = ReLU(0.54) = 0.54 、 h2 = ReLU(0.48) = 0.48 ，
輸出 = Sigmoid(0.612) = 0.648 → 再入院機率 64.8% 。

損失

若真實答案是「會再入院」，交叉熵損失 = −ln(0.648) = 0.433 。
這個數字就是訓練要最小化的目標。

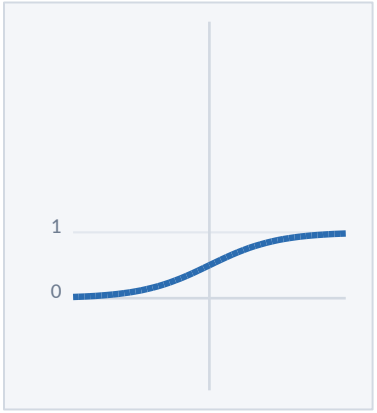
★ 複雜的不是單一步驟

是「有幾百萬個權重要調」。演算法本身很簡單。

★ 原生圖形，可直接在 PowerPoint 編輯

激活函數：非線性從哪裡來

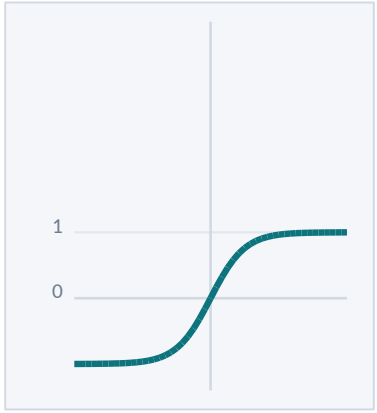
Sigmoid
1 / (1 + exp(−x))



輸出 0–1
★ 二分類的輸出層

梯度消失：兩端幾乎為 0

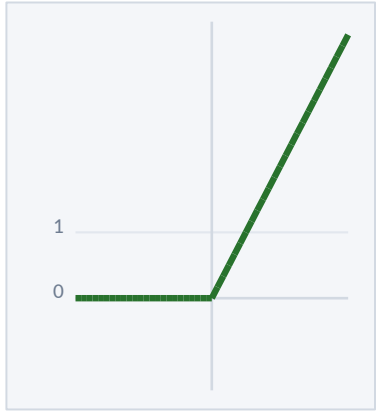
Tanh
tanh(x)



輸出 – 1–1
以 0 為中心

同樣有梯度消失問題

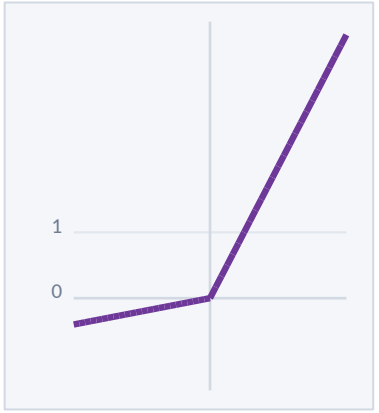
ReLU
max(0, x)



★ 隱藏層的預設選擇
計算快、不飽和

負區為 0 → 神經元可能死掉

Leaky ReLU
max(0.1x, x)



負區保留小斜率
解決神經元死亡

多一個超參數要調

為什麼一定要有激活函數：線性疊線性，還是線性



線性 → 線性 → 線性



線性 → ReLU → 線性

線性函數疊線性函數，還是線性函數。
沒有非線性， 100 層的網路和 1 層的效果完全一樣。

★ 激活函數就是深度學習能「深」下去的存在前提。

★ 為什麼一定要有

線性函數疊線性函數還是線性函數。
沒有非線性， 100 層和 1 層的效果完全一樣。

ReLU

隱藏層的預設選擇。計算快、不飽和。
缺點：負區完全為 0，神經元可能「死掉」。

Sigmoid

輸出 0–1，適合二分類的「輸出層」。
★ 不要用在隱藏層——會梯度消失。

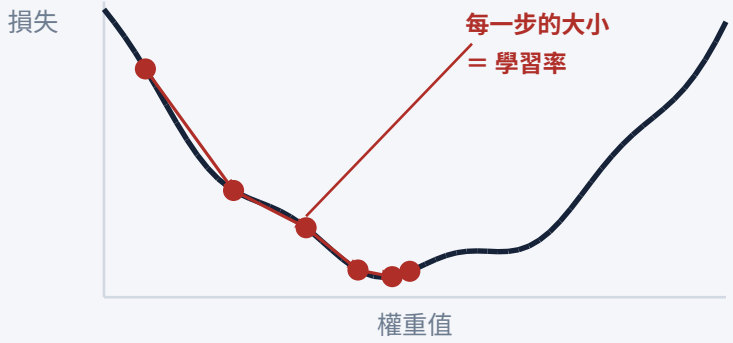
Leaky ReLU

負區保留小斜率，解決神經元死亡。
代價是多一個超參數。

原講義： RapidMiner_Deep_Learning_Architectures_v3.pdf ★原生圖形，可直接在 PowerPoint 編輯

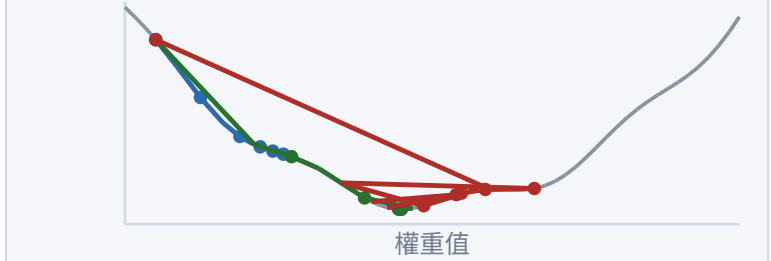
訓練在做什麼：梯度下降與反向傳播

梯度下降：往「下坡最陡」的方向走一步



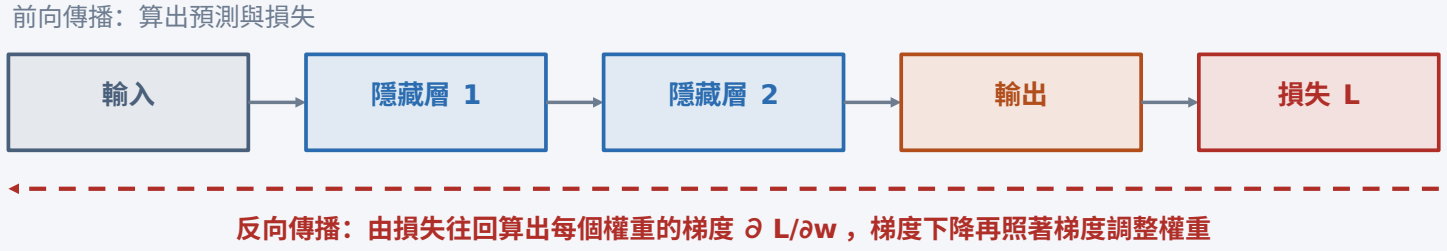
谷底就是損失最小的那組權重

學習率選錯會發生什麼



- 學習率太小（走太慢，訓練不完）
- ★ 學習率適中（穩定收斂）
- 學習率太大（跳過谷底，甚至發散）

反向傳播：用連鎖律把誤差一層層往回傳



梯度下降

站在損失曲面上，往「下坡最陡」的方向走一步。
每一步的大小就是學習率。

反向傳播

用連鎖律把「輸出的誤差」一層層往回傳，算出每個權重該調多少。

★ 學習率太小

走太慢，訓練不完。

★ 學習率太大

跳過谷底，甚至發散。 Loss 亂跳時，第一個該懷疑的就是學習率。

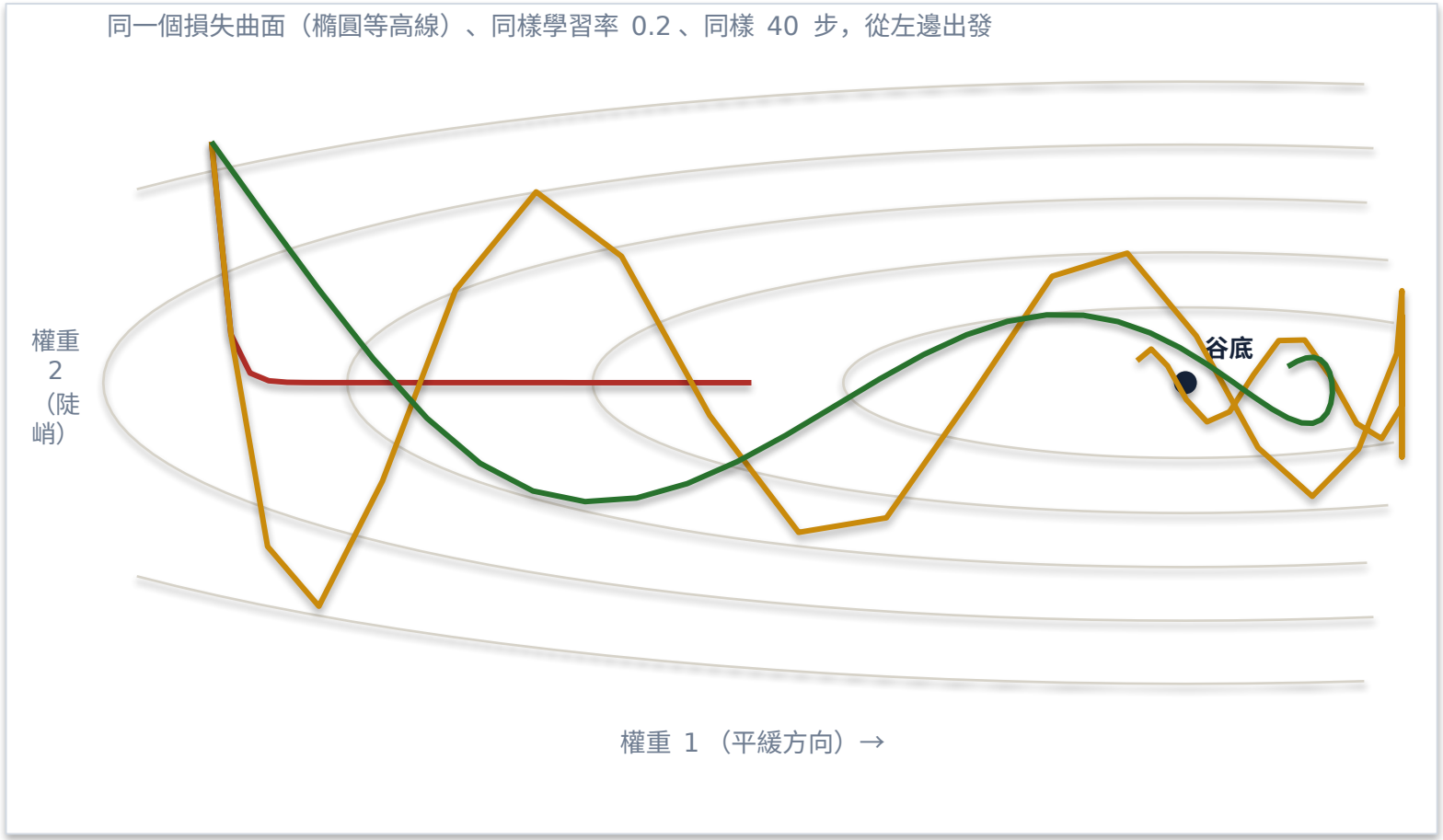
★ 原生圖形，可直接在 PowerPoint 編輯

優化器：梯度下降的幾種改良

優化器	改良了什麼	特點	什麼時候用
SGD	最基本的梯度下降	單純、需要仔細調學習率	研究上仍常用；配合學習率排程效果好
Momentum	加入「慣性」，累積過去的方向	能衝過小坑洞、加速收斂	SGD 的標準搭配
RMSProp	★ 每個參數各自調整學習率	對稀疏梯度友善	★ H2O 用的是同家族的 ADADELTA
Adam	Momentum ＋ RMSProp 的結合	★ 收斂快、幾乎不用調參	絕大多數情況的預設選擇

★ 實務建議：先用 Adam （學習率 0.001 ）跑通。只有在追求最後幾個百分點時，才考慮換 SGD ＋ 學習率排程慢慢磨。

實例： SGD 、 Momentum 、 Adam 走的路不一樣



SGD

40 步後離谷底 4.01 。陡峭方向很快到位，平緩方向卻幾乎沒動——單一學習率顧不了兩個方向。

Momentum

離谷底 0.47 ，這次最近。累積慣性在平緩方向加速，代價是會衝過頭、路徑來回擺盪。

Adam

離谷底 0.95 。每個參數各自調步伐，兩個方向一起穩定前進、不太需要細調學習率——所以它是預設選擇。

損失曲面： $0.05 \cdot w_1^2 + 2 \cdot w_2^2$ （一個方向平、一個方向陡，真實網路很常見）。學習率再加大， SGD 會在陡峭方向來回震盪甚至發散。

PART 03

Layers 設計

Designing the Network

在 AI Studio 裡，設計網路就是拖拉這些層。

看懂每一層在做什麼，介面上的每個選項就都有意義了。

一個典型網路的層堆疊



★ AI Studio 的 Deep Learning 運算子就是在拖拉這些層。看懂這張圖，介面上每個選項就都有意義了。

Dense 全連接層

主力運算層。神經元數是你要調的主要參數。

Activation 激活

引入非線性。隱藏層用 ReLU。

Dropout

訓練時隨機關掉一部分神經元，防過擬合。
★ 預測時自動關閉，不用你處理。

★ 逐層收窄

通常一層比一層窄（64 → 32 → 1），
逼模型把資訊壓縮成更精煉的表徵。

原講義： RapidMiner_Deep_Learning_Architectures_v3.pdf ★原生圖形，可直接在 PowerPoint 編輯

輸出層：最容易設錯的一層



二分類

1 個神經元 ＋ Sigmoid ＋ Binary Cross-Entropy。

多分類

N 個神經元 ＋ Softmax ＋ Categorical Cross-Entropy。
★ 各類機率加總 ＝ 1。

★ 多標籤

N 個神經元 ＋ Sigmoid （不是 Softmax）。
一個病人可以同時有三個診斷。

迴歸

1 個神經元 ＋ 不用激活 ＋ MSE ／ MAE。

★ 原生圖形，可直接在 PowerPoint 編輯

神經元數量與層數怎麼決定

面向	經驗法則	太少會怎樣	太多會怎樣
層數	從 2–3 層隱藏層開始；表格資料很少需要超過 5 層	欠擬合，學不出複雜關係	梯度傳不回去、過擬合、訓練慢
每層神經元數	第一層約為特徵數的 1–2 倍，之後逐層減半	資訊瓶頸，前面的層白學	參數暴增、過擬合
整體參數量	★ 小型表格資料的粗略原則：參數量不要超過樣本數	—	★ 幾乎必定過擬合

★ 最後一列是最實用的檢查：1000 筆資料配一個 50 萬參數的網路，那不是在學習，是在背答案。

實例：疊一個網路，算一算參數量



總參數量 = 576 + 2,080 + 33 = 2,689

公式

每一層的參數 = 神經元數 ×（上一層的輸出數 + 1）。「+1」是每個神經元的偏差項。Dropout、激活函數都沒有參數。

對照樣本數

若只有 1,000 筆訓練資料，參數是樣本的 2.7 倍——小型表格資料幾乎必定過擬合。先減層數或每層神經元。

經驗法則

從 2–3 層隱藏層開始；第一層約為特徵數的 1–2 倍，之後逐層減半；表格資料很少需要超過 5 層。

8 個特徵 → 64 → 32 → 1 的二分類網路（30 天再入院）。網站版可以加減層數、調特徵數與樣本數，即時看參數量與「參數 ÷ 樣本」的比例。

訓練前必做的兩件事（做錯會整個訓練不起來）

① 特徵標準化 —— 這不是可選項

- ★ 神經網路對輸入尺度極度敏感：年齡 0–100 與檢驗值 0–500 混在一起，梯度會被大尺度的特徵主導
- 做法： Z-score 標準化（減平均、除標準差）
- ★ 標準化的參數只能用「訓練集」算，再套用到驗證與測試集——用全部資料算就是資料洩漏（ M1 教過）

② 權重初始化 —— 不能全設為 0

- 全設 0 → 所有神經元算出來完全一樣 → 永遠學不出不同的特徵
- 全設太大 → 激活函數飽和（ Sigmoid 兩端）→ 梯度消失
- ★ 現代作法： He 初始化（搭配 ReLU ）或 Xavier 初始化（搭配 Sigmoid/Tanh ），框架都是預設好的，你通常不用管——但要知道為什麼不能設 0

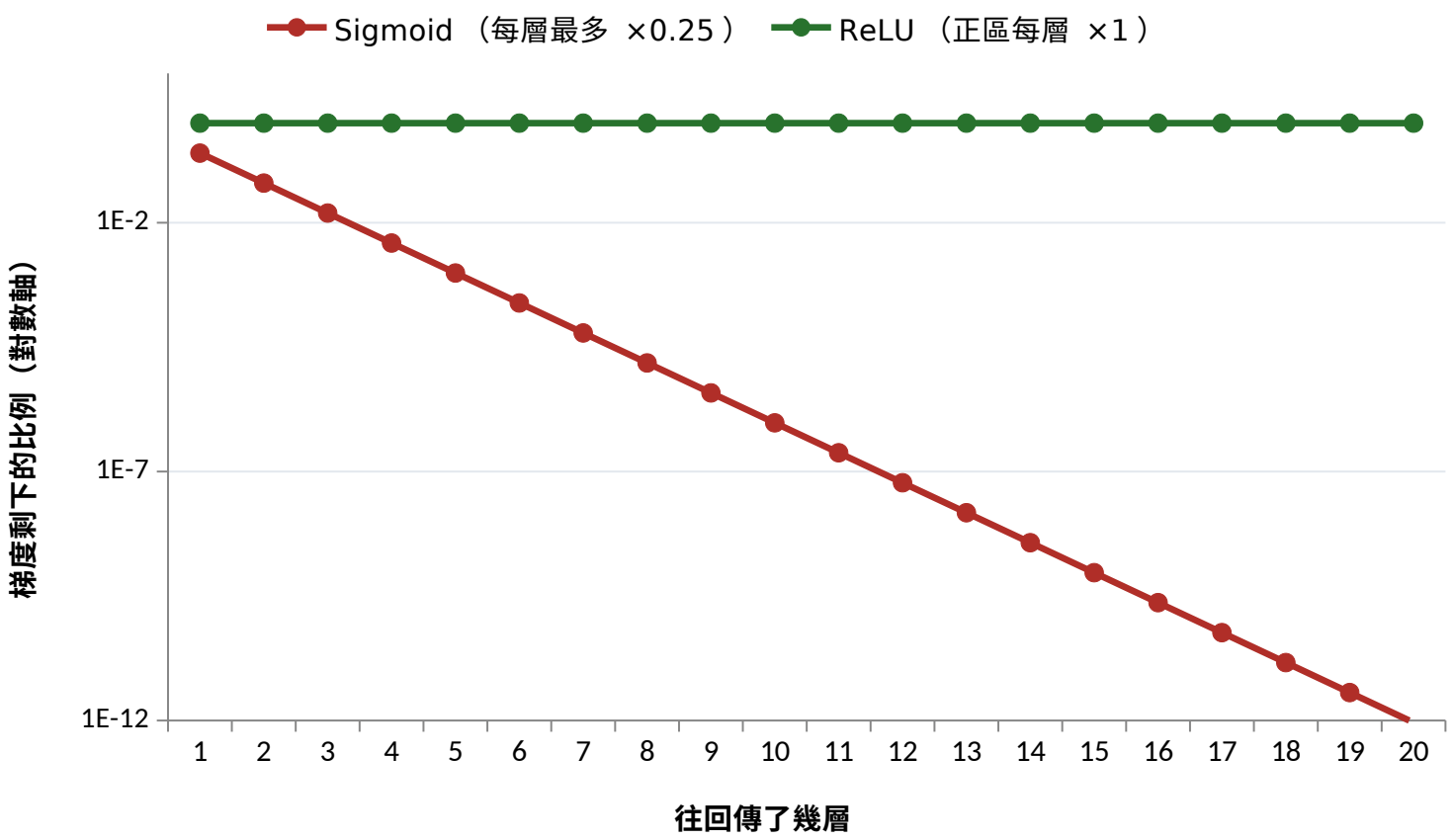
★ 這兩件事在 AI Studio 的 H2O 運算子裡是自動處理的。但用 DL4J 自訂架構時要自己確認。

梯度消失與梯度爆炸：症狀、成因、處方

問題	症狀	成因	處方
梯度消失 **Vanishing**	★ 訓練 Loss 幾乎不動； 越前面的層權重幾乎不變	Sigmoid ／ Tanh 在兩端的導數趨近 0， 連乘多層後梯度趨近 0	★ 改用 ReLU 、加殘差連接、 Batch Normalization
梯度爆炸 **Exploding**	Loss 突然變成 NaN 或極大值； 權重數值爆走	梯度連乘後數值急速放大， 常見於深層或循環網路	★ 梯度裁剪（Gradient Clipping）、 降低學習率、權重正則化

★ 一個實用的診斷法：訓練幾個 epoch 後，印出各層權重的變化量。若前面幾層幾乎沒變，就是梯度消失。

實例：梯度傳回第一層還剩多少？



Sigmoid 的導數最大只有 0.25

往回傳 5 層剩 9.8e-04 ， 10 層剩 9.5e-07 ， 20 層剩 9.1e-13—— 前面幾層幾乎學不動。

ReLU 在正區導數 = 1

梯度可以原封不動地傳回去，所以隱藏層一律預設 ReLU 。

其他處方

殘差連接（ ResNet ，第 45 張）、 Batch Normalization 、合適的初始化。反過來，連乘後急速放大就是梯度爆炸：用梯度裁剪。

這是「最好的情況」： Sigmoid 在輸入 0 時導數才是 0.25 ，輸入偏離 0 越遠導數越小，實際消失得更快。這正是 2012 年以前深層網路訓練不起來的主因。

表格資料要用深度學習時：類別特徵怎麼處理

這是深度學習在表格資料上少數真正有優勢的地方

嵌入層（深度學習在表格資料上的真正優勢）

【問題】診斷碼（ICD-10）有上萬種，怎麼餵給神經網路?

【做法一】One-Hot 編碼（M1 教過的傳統做法）
I10 高血壓 → [1, 0, 0, 0, ..., 0] ← 一萬維的向量
E11 糖尿病 → [0, 1, 0, 0, ..., 0]
問題① 維度爆炸：一萬個欄位，絕大多數是 0
問題② ★ 完全沒有語意：I10 與 I11（其他高血壓）的距離，
跟 I10 與 S72（股骨骨折）的距離一模一樣 —— 但臨床上顯然不同

【做法二】嵌入層 Embedding Layer（深度學習的做法）
I10 高血壓 → [0.31, −0.22, 0.87, ..., 0.14] ← 例如 32 維
E11 糖尿病 → [0.28, −0.19, 0.91, ..., 0.11] ← 位置相近
S72 股骨骨折 → [−0.65, 0.44, −0.12, ..., 0.78] ← 位置很遠

★ 這 32 維的數值不是你設的，是模型在訓練中學出來的
★ 學到的結果是：常一起出現、對預測有相似影響的診斷，向量會靠近

【嵌入維度怎麼設】
經驗法則：min(50, 類別數的四次方根 × 6) 或直接取 類別數 ^0.25 × 常數
例：1000 種診斷 → 約 32 維；50 種科別 → 約 16 維

★ 這與 M6 的 Word2Vec、M7 的詞嵌入是同一個概念的不同應用：
把「離散的類別」變成「有距離關係的連續向量」。

★ 但要注意：這需要足夠的資料量才學得出有意義的嵌入。
資料只有幾千筆時，CatBoost 的類別處理（M4）通常更務實。

深度學習在表格資料上，什麼時候真的有機會

★ 有機會的三種情況

- 高基數類別特徵很多（診斷碼、藥品碼、醫師代碼）
→ 嵌入層能學出有意義的表徵
- 要做多任務學習（同時預測再入院、住院天數、費用）
→ 共用底層表徵，樹模型難以做到
- ★ 要與非表格資料融合（表格 + 影像 + 文字）
→ 只有神經網路能把多種輸入接在一起

多數情況仍該用樹模型

- 純數值與低基數類別特徵 → XGBoost ／ CatBoost
- 資料量在幾千到幾萬筆之間
- 需要特徵重要性與可解釋性
- ★ 沒有 GPU 、也沒有時間反覆調參
- 團隊裡沒有人熟悉深度學習的除錯

★ 第三種情況（多模態融合）是深度學習不可取代的優勢。若你的專題要同時用影像與病歷欄位，那就是它的主場。

PART 04

訓練過程與過擬合

Training & Overfitting

訓練深度學習模型時，你有 80% 的時間在看一張圖。
學會判讀它，比記住任何演算法細節都重要。

Epoch、 Batch、 Iteration： 三個最常搞混的詞



Batch （批次）	一次送進網路的樣本數	100	每算完一批就更新一次權重
Iteration （迭代）	權重更新的次數	1000 / 100 = 10 次／ epoch	一次前向 + 一次反向
Epoch （週期）	整個訓練集被掃過幾遍	設 30 → 共 300 次權重更新	★ 太少欠擬合、太多過擬合

★ batch_size 的取捨：小批次雜訊多但泛化較好、記憶體省；大批次穩定快速但可能收斂到較差的解。

Batch 批次

一次送進網路的樣本數。每算完一批就更新一次權重。

Iteration 迭代

權重更新的次數。 1000 筆 ÷ batch 100 = 10 次／ epoch 。

Epoch 週期

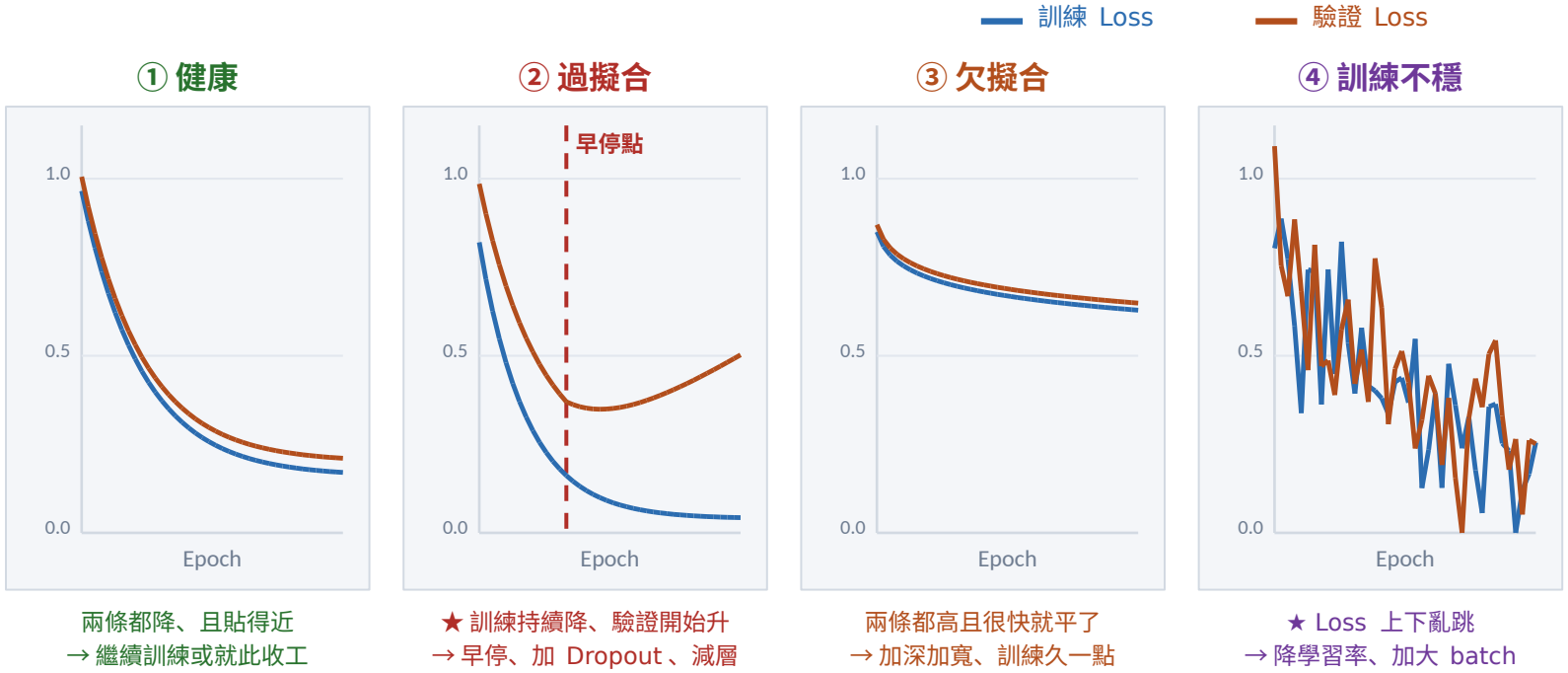
整個訓練集被掃過幾遍。★ 太少欠擬合、太多過擬合。

batch_size 的取捨

小批次雜訊多但泛化較好、記憶體省；
大批次穩定快速但可能收斂到較差的解。常見起點 32–128 。

★ 原生圖形，可直接在 PowerPoint 編輯

讀懂 Loss 曲線：四種樣態與處方



★ 訓練深度學習模型時，你有 80% 的時間在看這張圖。學會判讀它，比記住任何演算法細節都重要。

① 健康

兩條都降、且貼得近 → 繼續訓練或就此收工。

★ ② 過擬合

訓練持續降、驗證開始升 →
早停、加 Dropout 、減少層數。

③ 欠擬合

兩條都高且很快就平了 → 加深加寬、訓練久一點。

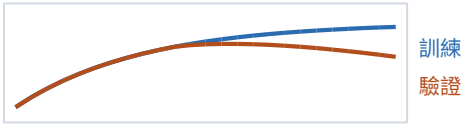
★ ④ 訓練不穩

Loss 上下亂跳 → 降學習率、加大 batch 。

原講義： RapidMiner_Deep_Learning_Architectures_v3.pdf ★原生圖形，可直接在 PowerPoint 編輯

對付過擬合的六招

症狀：訓練 Loss 一直降，驗證 Loss 開始升



① 早停 Early Stopping

驗證分數不再進步就停止訓練

★ 最有效、最便宜的一招
幾乎沒有副作用，一定要開

② Dropout

訓練時隨機關掉一部分神經元

強迫網路不能依賴特定神經元
常用 0.2–0.5；預測時自動關閉

③ 正則化 L1 ／ L2

在損失函數加上權重大小的懲罰

L2 讓權重平滑、 L1 讓權重稀疏
H2O 預設 L1 = 1e-5

④ 減少模型容量

層數減少、每層神經元減少

★ 資料少就別用大網路
這是最根本的解法

⑤ 增加資料

蒐集更多、或用資料增強

★ 效果最好但成本最高
影像可用旋轉／翻轉／裁切

⑥ Batch Normalization

把每層的輸出重新標準化

加速收斂、附帶輕微正則化
通常放在 Dense 與激活之間

★ 動手順序：先開早停（免費）→ 再減少模型容量（最根本）→ 才加 Dropout 與正則化（要調參）。

★ 動手順序很重要

先開早停（免費）→ 再減少模型容量（最根本）→
才加 Dropout 與正則化（要調參）。

早停 Early Stopping

最有效也最便宜。幾乎沒有副作用，一定要開。

Dropout

常用 0.2–0.5 。強迫網路不能依賴特定神經元。

★ 減少模型容量

資料少就別用大網路。這是最根本的解法，卻最常被忽略——大家總想先加 Dropout 。

★ 原生圖形，可直接在 PowerPoint 編輯

H2O 深度學習的實際參數（AI Studio 內建）

對照原講義的技術說明文件，逐項解釋在做什麼

原講義： 1. H2O 深度學習流程的技術性細節說明 .docx

```
【網路架構】
 hidden = [50, 50]                  兩層隱藏層，每層 50 個神經元
 activation = Rectifier             即 ReLU， max(0, x)
 loss_function = Automatic          依標籤型別自動選（分類→交叉熵、迴歸→ MSE ）

【訓練與優化】
 rate = 0.005                       初始學習率（ adaptive_rate=true 時不生效）
 adaptive_rate = true               ★ 啟用 ADADELTA，每個參數自動調學習率
   rho = 0.99                         指數加權平均的衰減係數
   epsilon = 1e-8                     防止除以零的極小值
 rate_annealing = 1e-6              學習率逐步下降（僅手動學習率時生效）
 nesterov_accelerated_gradient = true    比一般 momentum 更早預測方向（同上）
 epochs = 10                        整個訓練集掃描 10 次
 train_samples_per_iteration = -2        自動決定每次迭代的樣本數

【正則化】
 l1 = 1e-5                          ★ 絕對值懲罰 → 讓部分權重趨近 0 （稀疏化）
 l2 = 0.0                           未啟用；若啟用則以平方和抑制權重爆炸
 max_w2 = 10.0                      限制單一神經元的權重平方和，提升數值穩定

【資料前處理】
 standardize = true                 ★ 所有數值特徵做 Z-score 標準化
 missing_values_handling = MeanImputation    數值欄缺值以平均數填補
```

★ 注意最後兩行：這是 H2O 自動幫你做的。但「用平均數補缺值」是否恰當，
要看你的資料（回想 M1 的 MCAR/MAR/MNAR ）—— 別讓自動化替你做決定。

AI Studio 的三種神經網路實作

實作	特點	適合	限制
Neural Net （內建）	最陽春的多層感知器；設定簡單	教學示範、快速試水溫	功能少，無 Dropout 等現代機制
H2O Deep Learning	★ 參數完整、自動標準化與缺值處理； CPU 即可跑	★ 表格資料的深度學習主力	不支援卷積，做不了影像
Deep Learning （DL4J）	支援 Layer 逐層堆疊、 CNN 、 GPU	★ 影像任務、需要自訂架構時	設定複雜，需理解每一層

★ 課程安排： W8 用 H2O 建立直覺（表格資料）， W9 用 DL4J 做 CNN （MNIST 影像）。

實作 1 ｜ Iris 與 Titanic：逐步加深網路

【AI Studio 範例】 Repository ▸ 8. Deep Learning/1 、 2.1 、 2.2 、 3 、 4

開啟檔案

```
Repository ▸ 8. Deep Learning/
  1 Deep Learning for simple
    classification - Iris.rmp
  2.1 Fully-connected and
    output Layers.rmp
  2.2 monitor the performance
    metrics.rmp
  3 various Layers Titanic.rmp
  4 No. of Neurons.rmp
```

預期結果

- ▸ 層數加深不一定變好， Titanic 這種小表格資料尤其明顯
- ▸ 深網路的訓練分數會很漂亮，驗證分數卻下降
- ▸ Dropout 能拉近兩者差距
- ▸ ★ 梯度提升樹（GBT）在 Titanic 上很可能贏過所有深度學習設定

操作步驟

1. 先跑 Iris 範例，記錄準確率——這是最單純的三分類
2. ★ 打開 2.2 ，觀察訓練過程的 Loss 曲線長什麼樣
3. 在 Titanic 上先跑 1 層 10 個神經元，記錄訓練與驗證分數
4. 改成 2 層 [50,50] ，比較兩者的差距
5. 改成 4 層 [200,200,200,200] ，★ 觀察是否出現過擬合
6. 在過擬合的那個設定上加 Dropout 0.3 ，再跑一次
7. ★ 最後：用 M4 的 Gradient Boosted Trees 跑同一份 Titanic ，比較誰贏

PART 05

W9　卷積神經網路 CNN

Convolutional Neural Networks

為什麼影像不能直接丟進全連接網路?

因為 224×224 的彩色影像展平之後是 15 萬個輸入——參數會爆炸。

全連接網路處理影像的三個問題

① 參數量爆炸

- 224×224×3 的影像展平 = 150,528 個輸入
- 接一層 1000 個神經元 → 1.5 億個權重，只是第一層
- ★ 完全無法訓練，也一定過擬合

② 丟失空間結構

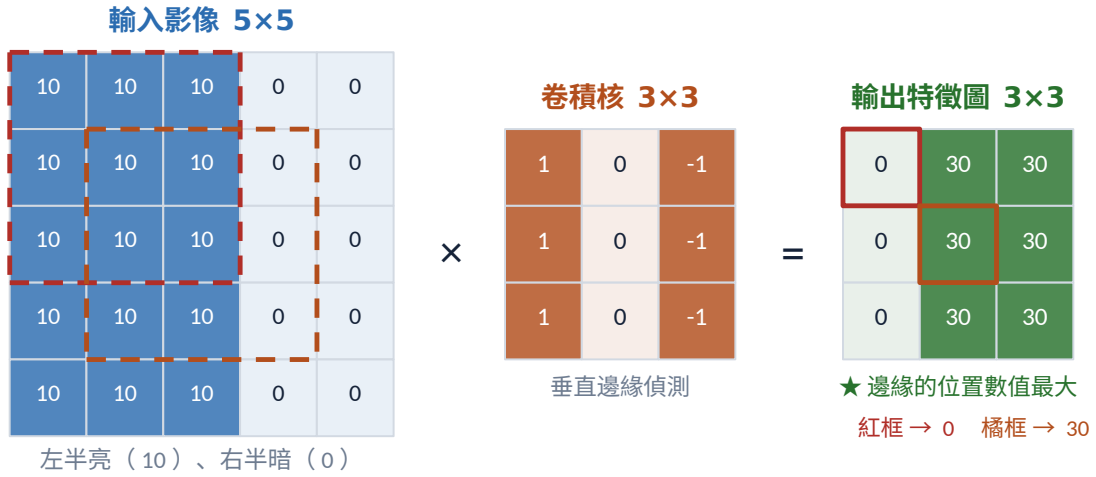
- 展平之後，「相鄰的像素」與「對角的像素」變得一樣遠
- ★ 但影像的意義正是來自空間關係——這等於把資訊丟掉了

③ 沒有平移不變性

- 病灶出現在左上角和右下角，對全連接網路是完全不同的輸入
- 要學會「同一個東西出現在不同位置」，需要指數級的資料量

★ 卷積一次解決這三個問題：權重共享（省參數）、局部連接（保留空間）、滑動視窗（平移不變）。

卷積運算：一步一步算給你看



左上角（紅框）怎麼算

取影像左上 3×3 區塊：

10 10 10 / 10 10 10 / 10 10 10

與卷積核逐格相乘再相加：

10×1 + 10×0 + 10×(−1) = 0

三列都是 0 → 總和 = 0

★ 左半均勻，沒有邊緣 → 0

輸出尺寸公式：(輸入 − 卷積核 + 2×padding) / stride + 1

- 本例：(5 − 3 + 0) / 1 + 1 = 3
- 若 padding = 1：(5 + 2 − 3) / 1 + 1 = 5　→ 尺寸不變（最常用的設定）
- 若 stride = 2：(5 − 3) / 2 + 1 = 2　→ 尺寸減半（用來降維）

★ 關鍵理解：卷積核的數值不是你設的，是模型「學」出來的 —— 它會自己學出邊緣、紋理、形狀偵測器。

在做什麼

卷積核在影像上滑動，每個位置做「逐格相乘再相加」。

本例

垂直邊緣偵測核。左半均勻的地方輸出 0，邊緣所在的位置輸出 30。

輸出尺寸公式

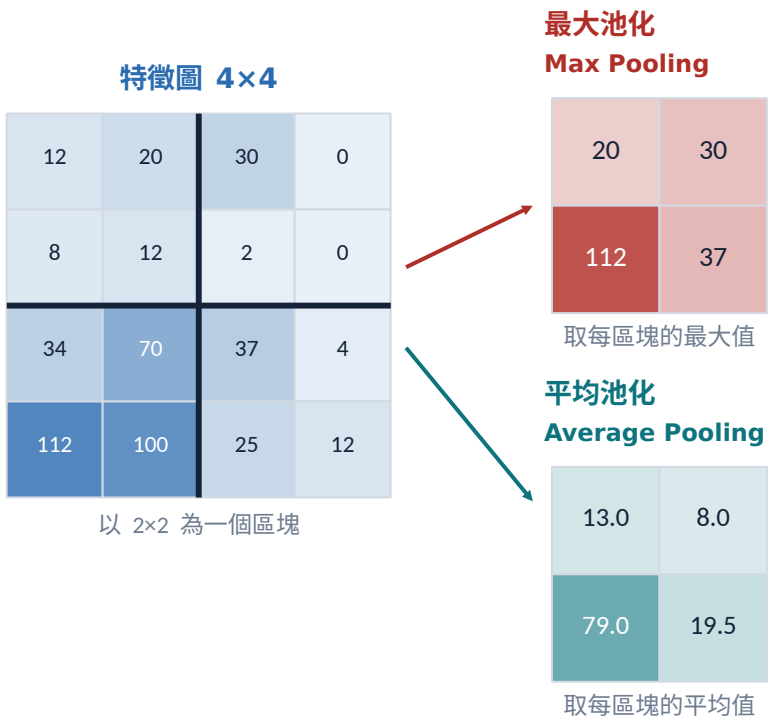
(輸入 − 卷積核 + 2×padding) / stride + 1。
本例 (5−3)/1+1 = 3。

★ 關鍵理解

卷積核的數值不是你設的，是模型「學」出來的。
它會自己學出邊緣、紋理、形狀等偵測器。

★ 原生圖形，可直接在 PowerPoint 編輯

池化 Pooling：把特徵圖縮小



池化在做什麼、為什麼要做

- 降低尺寸： 4×4 → 2×2 ，參數量與計算量大減
- ★ 平移不變性：物體稍微移動，結果仍相近
- 擴大感受野：後面的層能看到更大範圍
- 沒有參數要學，純粹是固定運算

最大池化 vs 平均池化：

- 最大池化保留「最強的訊號」→ 影像分類主流
- 平均池化較平滑 → 常用在網路最末端

★ 近年趨勢：有些架構改用 stride = 2 的卷積取代池化，讓降維也變成可學習的

★ 醫學影像要小心：病灶的「精確位置」有時很重要，過度池化會把位置資訊丟掉。

做什麼

把每個小區塊取最大值（或平均值），尺寸減半。

為什麼要做

降低參數量、擴大感受野、
★ 提供平移不變性（物體稍微移動，結果仍相近）。

最大 vs 平均

最大池化保留最強訊號 → 影像分類主流。
平均池化較平滑 → 常用在網路末端。

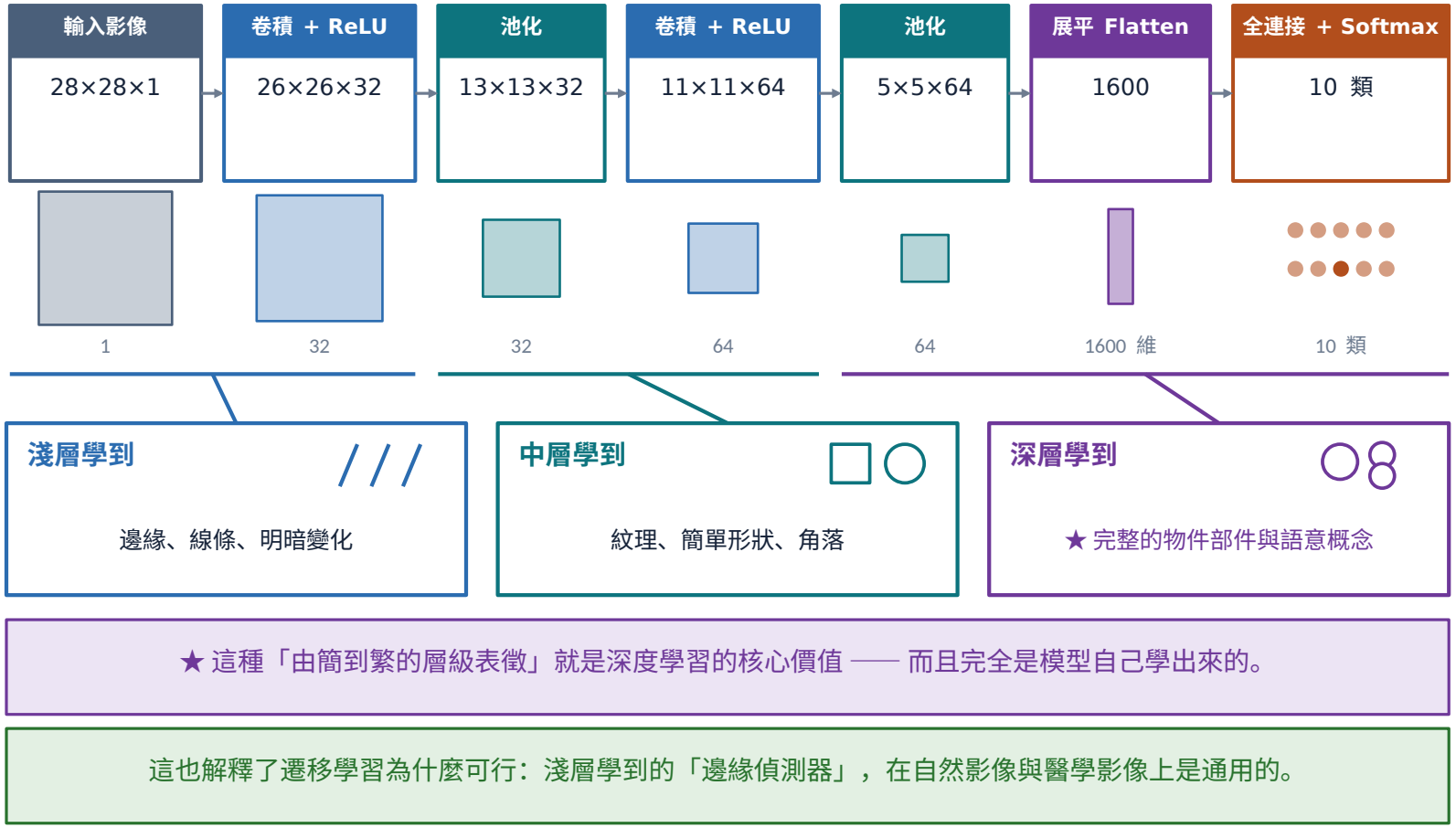
★ 醫學影像要小心

病灶的「精確位置」有時很重要，
過度池化會把位置資訊丟掉。

★ 原生圖形，可直接在 PowerPoint 編輯

CNN 完整流程與層級特徵

以 MNIST 手寫數字為例



流程

卷積 → 池化 → 卷積 → 池化 → 展平 → 全連接 → 輸出。

★ 層級表徵

淺層學邊緣線條、中層學紋理形狀、深層學完整物件部件。
完全是模型自己學出來的。

尺寸怎麼變

空間尺寸逐步縮小（28→26→13→11→5），

但通道數逐步增加（1→32→64）。

★ 這解釋了遷移學習為什麼可行

淺層的「邊緣偵測器」，
在自然影像與醫學影像上是通用的。

★ 原生圖形，可直接在 PowerPoint 編輯

CNN 的關鍵超參數

參數	作用	常見設定	調錯會怎樣
卷積核大小 kernel	一次看多大的範圍	3×3 （現代主流）	太大 → 參數多、細節丟失；太小 → 感受野長太慢
卷積核數量 filters	學幾種不同的特徵	32 → 64 → 128 逐層加倍	太少 → 表徵能力不足；太多 → 過擬合、慢
步幅 stride	滑動的間隔	1 （保留細節）或 2 （降維）	太大 → 跳過細節
補零 padding	邊緣是否補零	same （尺寸不變）最常用	不補 → 每層都縮小，深層會不夠用
池化大小 pool	一次縮小多少	2×2	太大 → 位置資訊丟太多

★ 現代趨勢： 3×3 卷積堆疊取代大卷積核（ VGG 的貢獻）； 有些架構用 stride=2 的卷積取代池化，讓降維也可學習。

PART 06

預訓練模型與遷移學習

Pre-trained Models & Transfer Learning

你不會從零訓練一個 CNN。

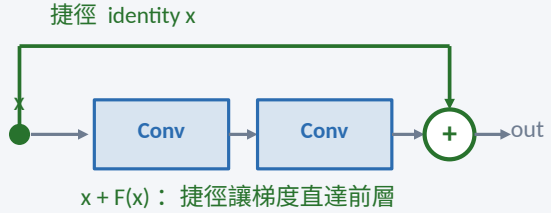
在醫學影像領域，這幾乎是唯一可行的路。

預訓練 CNN 模型家族

原講義： deep_learning_networks_explained.pptx

SimpleCNN	VGG16 / VGG19	ResNet50	SqueezeNet	Xception
自建小網路	小卷積核堆疊	★ 殘差捷徑	Fire Module 壓縮	空間與通道分離卷積
3–5 層	16 ／ 19 層	50 層	少參數	深且高效
教學與小型任務的基準	通用影像特徵抽取器 經典但笨重	解決深層訓練困難 最常用的骨幹	模型體積小 適合部署到邊緣裝置	效率與準確度兼顧

殘差連接（ResNet）解決了什麼



- 問題：網路太深時，梯度往回傳的路上會逐漸消失 → 前面幾層根本學不動
- 解法：加一條「捷徑」讓輸入直接跳過幾層加到輸出（x + F(x)）

★ 影響：從此網路可以做到上百層，是 2015 年之後多數視覺架構的基礎

★ 對本課程的意義：你不會從零設計架構，而是「選一個預訓練模型 + 遷移學習」。

VGG 系列

小卷積核堆疊。經典、好理解，但參數多、笨重。

★ ResNet

殘差捷徑（x + F(x)）讓梯度可以沿捷徑直達前層。
從此網路能做到上百層—— 2015 年之後所有視覺架構的基礎。

SqueezeNet

Fire Module 壓縮模型體積，適合部署到邊緣裝置。

★ 對本課程的意義

你不會從零設計架構，
而是「選一個預訓練模型 + 遷移學習」。選型比架構設計重要。

★ 原生圖形，可直接在 PowerPoint 編輯

PART 07

經典 CNN 架構與設計理由

Classic Architectures & Design Rationale

這些模型你都能在 AI Studio 的匯入清單裡直接選到。
但選哪一個、為什麼——這一節把每個架構的設計動機講清楚。

讀架構圖時該看什麼

★ 重點不在「層數」，在「資料怎麼被轉換」

- 從像素 → 邊緣 → 紋理 → 部件 → 語意，每一段由哪些層負責
- 空間尺寸怎麼縮小、通道數怎麼增加
- 全連接層在哪裡接手

每個架構都是在解決前一代留下的問題

- VGG 解決：大卷積核參數太多、架構雜亂
- ResNet 解決： VGG 太深就訓練不動（退化問題）
- SqueezeNet ／ Xception 解決：模型太肥，塞不進裝置

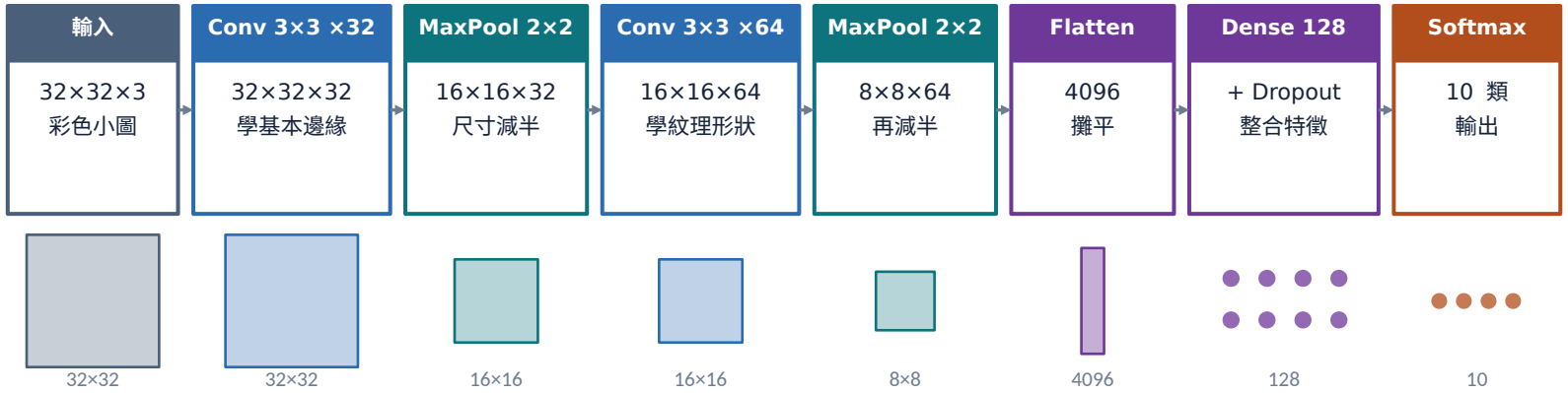
三個評估面向

- 表現：準確度（但要在同一份資料上公平比較）
- 成本：參數量、檔案大小、推論速度
- ★ 適配度：預訓練的資料域與你的任務有多接近

原講義： deep_learning_networks_explained.pptx

SimpleCNN： 先看懂最基本的架構

架構：卷積 ↔ 池化交替 → 展平 → 全連接 → 輸出



為什麼這樣設計：四個規律，所有 CNN 都遵守

① 空間縮小、通道變多	32×32×3 → 8×8×64 。位置資訊逐步換成語意資訊
② 卷積之後接池化	卷積抽特徵、池化降維，成對出現
③ 通道數通常逐層加倍	32 → 64 → 128 。越深要辨識的特徵越多樣
④ 最後才用全連接	★ 卷積層負責「看」，全連接層負責「決定」。分工明確

參數量約 0.1M 、檔案 < 1 MB —— 這是你第一個該親手跑通的 CNN 。

★ 用途：教學與小型任務的基準模型。任何專題都該先跑一個 SimpleCNN 當基準線，
再去試 VGG 、 ResNet —— 否則你不知道複雜架構到底帶來多少改善。

四個規律，所有 CNN 都遵守

① 空間縮小、通道變多　② 卷積之後接池化
③ 通道數逐層加倍　④ 最後才用全連接

★ 分工明確

卷積層負責「看」，全連接層負責「決定」。

尺寸怎麼變

32×32×3 → 8×8×64 。
位置資訊逐步換成語意資訊。

★ 用途

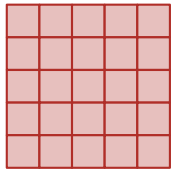
教學與小型任務的基準模型。任何專題都該先跑它當基準線，
否則你不知道複雜架構到底帶來多少改善。

★ 原生圖形，可直接在 PowerPoint 編輯

VGG：為什麼堅持只用 3×3 卷積

核心主張：與其用一個大卷積核，不如堆疊多個 3×3 —— 感受野完全相同

做法 A：一個 5×5 卷積



感受野 5×5

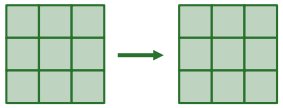
參數 = 5×5×64×64

= 102,400

非線性次數： 1 次

（只有一層 ReLU）

★ 做法 B：兩個 3×3 卷積堆疊



第 1 層 3×3　第 2 層 → 等效 5×5

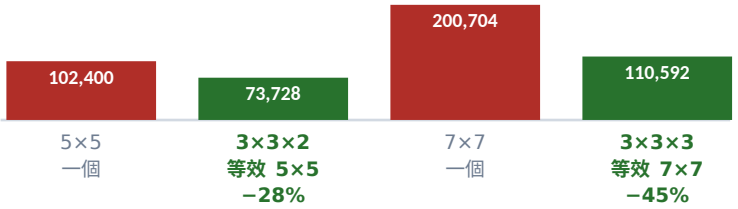
參數 = 2×(3×3×64×64)

= 73,728

非線性次數： 2 次

（兩層都有 ReLU）

參數量對比（數字已驗算）



VGG16 共 138M 參數



堆疊帶來的三個好處

① 參數更少：等效 5×5 少 28%、等效 7×7 少 45%

② ★ 非線性更多：每層之間都有 ReLU，同樣的感受野卻能表達更複雜的函數

③ 架構規則整齊：只用一種卷積核，好實作也好調整

★ 代價：全連接層太肥。VGG16 模型檔案約 528 MB —— 邊緣裝置或行動端幾乎不可用。

後續架構（ResNet 之後）多改用全域平均池化取代全連接層，就是為了解決這個問題。

核心主張

與其用一個 5×5，不如堆兩個 3×3—— 感受野相同。

★ 好處一：參數更少

2×(3×3×64×64) = 73,728 vs 5×5 的 102,400，少 28%。
等效 7×7 時（三個 3×3）少 45%。

★ 好處二：非線性更多

每層之間都有 ReLU。同樣的感受野，卻能表達更複雜的函數。

★ 代價

全連接層太肥： 138M 參數中約 90% 集中在最後三層，
檔案 528 MB，邊緣裝置幾乎不可用。

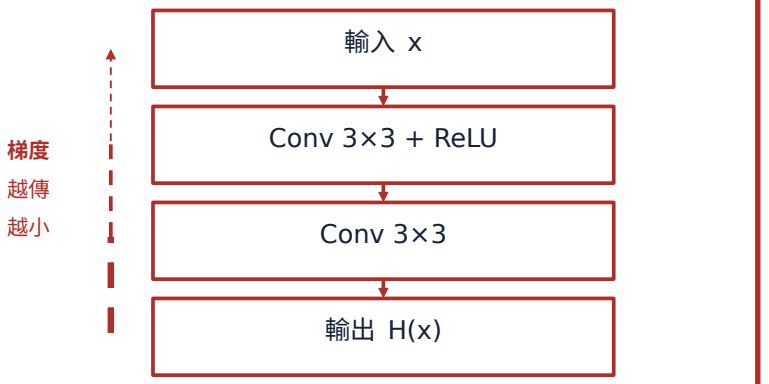
VGG 家族的三個預訓練版本：差在「用什麼資料訓練」

版本	預訓練資料	學到什麼樣的特徵	什麼時候選它
VGG16-ImageNet	ImageNet 自然影像 1000 類、 120 萬張	通用視覺特徵：邊緣、紋理、物件部件	★ 遷移學習的通用起點；與醫學影像領域差異大但泛化性好
VGG16-CIFAR10	CIFAR-10 32×32 小圖、 10 類	適合小尺寸影像的特徵	教學示範；輸入是小圖時較貼近
VGG16-VGGFace	人臉資料集	★ 臉部身分、表情、局部差異	人臉相關任務；★ 醫療場域涉及人臉辨識時要格外注意個資

★ 這張表要傳達的觀念：同樣是 VGG16，預訓練資料不同，學到的特徵就完全不同。選模型時「權重來源」跟「架構」一樣重要。

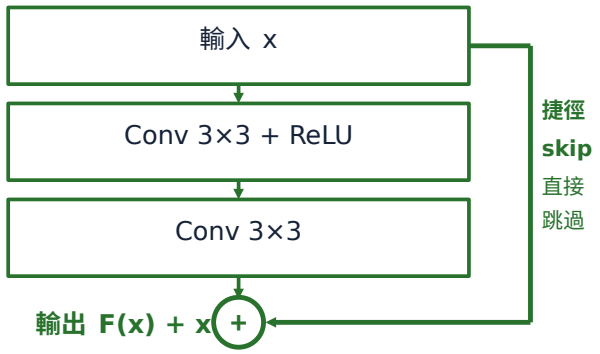
ResNet：殘差連接解決了什麼

一般深層網路：層層相疊



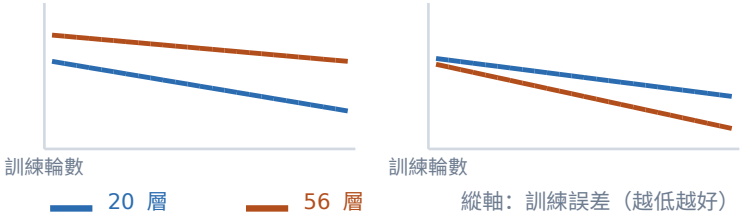
★ 梯度傳不回前層 → 深的反而比淺的差（退化問題）

★ 殘差塊：加一條捷徑



★ 梯度可沿捷徑直達前層 → 上百層仍能穩定訓練

一般網路：56 層比 20 層還差　　**★ ResNet：越深越好**



為什麼「學修正量」比「學完整映射」容易

- 一般網路要學 H(x)：從輸入直接學出完整的輸出映射
- 殘差網路只學 F(x) = H(x) − x：只需學「該修正多少」

★ 若這層其實不需要做什麼，F(x) 學成 0 就好 ——
這比學成「恆等映射」容易得多

★ ResNet50 只有 25.6M 參數，卻勝過 138M 的 VGG16

★ 用途：目前最常用的視覺骨幹模型。醫學影像遷移學習的預設起點。

參數量與檔案大小（ 98 MB ）都在可接受範圍，準確度與訓練穩定性也好 —— 這是它成為主流的原因。

問題：退化

層數變多時梯度傳不回前層，深的反而比淺的差。
這不是過擬合——連訓練分數都變差。

★ 解法：學修正量

一般網路學 H(x)；殘差網路學 F(x) = H(x) − x。
若這層不需要做什麼，F(x) 學成 0 就好——比學成恆等映射容易得多。

結果

從此網路可做到上百層。 ResNet50 只有 25.6M 參數，
準確度卻明顯優於 138M 的 VGG16。

★ 用途

目前最常用的視覺骨幹。
醫學影像遷移學習的預設起點。

原講義： deep_learning_networks_explained.pptx　★ 原生圖形，可直接在 PowerPoint 編輯

兩種「把參數變少」的設計思路

SqueezeNet： Fire Module

輸入 128 通道

Squeeze： 1×1 卷積 → 16 通道

Expand 1×1 → 64

Expand 3×3 → 64

合併 → 128 通道

參數： 2,048 ＋ 1,024 ＋ 9,216 = 12,288

一般 3×3 卷積要 147,456 →★少 91.7%

設計理由：先用 1×1 把通道壓下來，再展開

Xception： 深度可分離卷積

標準卷積：一次做完「空間」與「通道」

3×3×256×256 = 589,824 個參數

拆成兩步

① Depthwise

每個通道各自做 3×3

3×3×256 = 2,304

② Pointwise

用 1×1 混合通道

256×256 = 65,536

合計 67,840 個參數

★比標準卷積少 88.5%（約 8.7 倍），效果卻相近

設計理由：空間關聯與通道關聯其實可以分開處理

—— 這個洞見後來也被 MobileNet 等行動端架構採用

參數量對比（同樣的輸入／輸出通道數）

一般 3×3 卷積（基準）	147,456	
★ Fire Module （SqueezeNet）	12,288	－91.7%
標準卷積（基準）	589,824	
★ 深度可分離卷積（Xception）	67,840	－88.5%

★ 為什麼要在意參數量：模型要放進手機、平板、床邊裝置時， 528 MB 的 VGG16 根本塞不進去。

SqueezeNet 的 Fire Module

先用 1×1 把 128 通道壓到 16，再展開回 128。
參數 12,288 vs 一般卷積的 147,456——★少 91.7%。

Xception 的深度可分離卷積

把「空間」與「通道」拆成兩步：
depthwise 2,304 ＋ pointwise 65,536 = 67,840。

★ 對比標準卷積

589,824 → 67,840，少 88.5%（約 8.7 倍），效果卻相近。

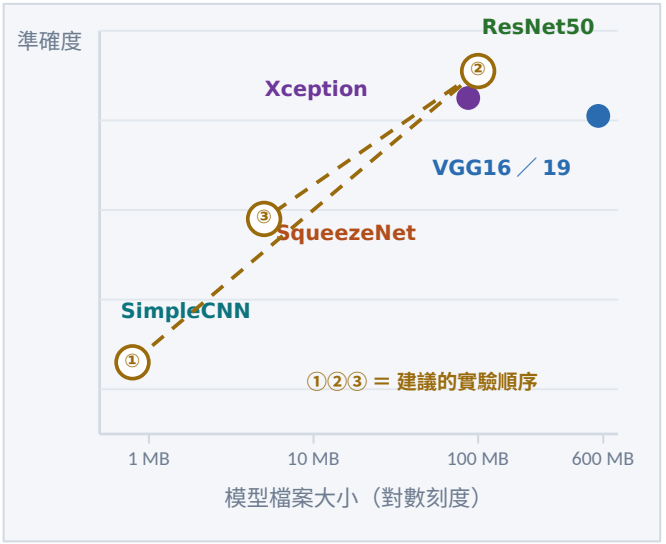
★ 為什麼要在意參數量

模型要放進手機、平板、床邊裝置時，
528 MB 的 VGG16 根本塞不進去。

六個經典架構的選型對照

模型	參數量	檔案	最適合的用途
SimpleCNN	~0.1M	< 1 MB	★ 教學、基準線、小型任務
VGG16	138M	528 MB	通用特徵抽取；★ 但太肥
VGG19	144M	549 MB	表徵略強，過擬合風險也升高
ResNet50	25.6M	98 MB	★ 醫學影像遷移學習的預設起點
SqueezeNet	1.2M	5 MB	★ 邊緣裝置、行動端部署
Xception	22.9M	88 MB	效率與準確度兼顧

模型大小 vs 準確度：沒有最好，只有最適合



★ 課程建議的實驗順序：SimpleCNN 建立基準線 → ResNet50 遷移學習 → 有部署限制時才考慮 SqueezeNet。

VGG 仍列出來，是因為它的設計思想（3×3 堆疊）影響深遠、也最好理解 —— 但實務上已很少直接使用。

★ 公平比較的作法：同一份資料、同樣的訓練輪數與資料增強，只換架構。否則比較沒有意義。

選型的三個問題：① 資料域接近嗎？ ② 資料量夠嗎？ ③ 要部署在哪裡？

★ 課程建議的實驗順序

SimpleCNN 建立基準線 → ResNet50 遷移學習 →
有部署限制時才考慮 SqueezeNet。

VGG 為什麼還列出來

它的設計思想（3×3 堆疊）影響深遠、也最好理解，
但實務上已很少直接使用。

★ 公平比較的作法

同一份資料、同樣的訓練輪數與資料增強，
只換架構。
否則比較沒有意義。

選型的三個問題

① 資料域接近嗎？ ② 資料量夠嗎？ ③ 要部署在哪裡？

★ 原生圖形，可直接在 PowerPoint 編輯

架構設計思想的演進：每一代解決了什麼

年代	架構	核心創新	解決的問題	留下的問題
2012	AlexNet	以 GPU 訓練深層 CNN 一戰成名 ＋ ReLU ＋ Dropout	證明深度學習在影像上可行	架構隨意，缺乏設計原則
2014	VGG	★ 統一用 3×3 卷積堆疊	架構規則化、參數效率提升	全連接層太肥；太深就訓練不動
2015	ResNet	★ 殘差捷徑 x + F(x)	★ 深度不再是障礙，可達上百層	仍有相當的運算量
2016	SqueezeNet	Fire Module （1×1 壓縮）	體積比 AlexNet 小 50 倍	準確度略低
2017	Xception	深度可分離卷積	運算量大減、準確度維持	實作較複雜
2020+	Vision Transformer	把影像切塊當序列，用注意力	★ 大資料下超越 CNN	★ 小資料上仍常輸給 CNN

★ 注意最後一列：Vision Transformer 在大資料上很強，但醫學影像資料通常很少——這種情況下 CNN ＋ 遷移學習仍是更務實的選擇。這與 M5 一貫的立場一致：新的不一定適合你。

實作 4 ｜同一資料集比較四種架構

【AI Studio】模型匯入 → 選擇預訓練架構

開啟檔案

```
Repository ▸ 8. Deep Learning/
  5 CNN MNIST.rmp （改架構用）
  ML. Deep Learning for image
    classification - operational.rmp

★ AI Studio 的模型匯入清單中
  可直接選到 SimpleCNN、VGG16、
  ResNet50、SqueezeNet、Xception
```

預期結果

- 遷移學習的訓練時間遠短於從零訓練
- ★ ResNet50 通常準確率最高
- SqueezeNet 準確率略低，但檔案小 20 倍以上
- ★「最準」與「最適合」往往不是同一個

操作步驟

1. 固定資料集、訓練輪數、資料增強設定——只換架構
2. ① SimpleCNN 從零訓練，記錄準確率、訓練時間、模型大小
3. ② VGG16-ImageNet 遷移學習（凍結卷積層）
4. ③ ResNet50-ImageNet 遷移學習（凍結卷積層）
5. ④ SqueezeNet 遷移學習
6. ★ 做成表：準確率／訓練時間／模型大小／推論速度
7. ★ 寫下：如果要部署到平板上，你會選哪一個？為什麼？

遷移學習：不要從零開始訓練



① 特徵抽取

凍結全部卷積層，只訓練最後的分類層。
★ 資料 < 1,000 張時的首選，分鐘級就能跑完。

② 部分微調

凍結淺層、解凍深層。
★ 資料 1,000–10,000 張時最常用的折衷。

③ 全部微調

全部解凍，用小學習率重訓。
資料 > 10,000 張且領域差異大時。

★ 選擇依據

資料量越少、與預訓練領域越相近 → 凍結越多層。
醫學影像資料通常很少，多半停在①或②。

★ 原生圖形，可直接在 PowerPoint 編輯

遷移學習的三個實務細節

① 預訓練來源會影響結果

- ImageNet 預訓練（自然影像）→ 泛化性較好，是通用起點
- 醫學影像專用預訓練 → 資料極少時表現較佳
- ★ 研究顯示（RadImageNet， Mei et al., 2022）：資料少時醫學影像預訓練常較佳，資料多時差距縮小

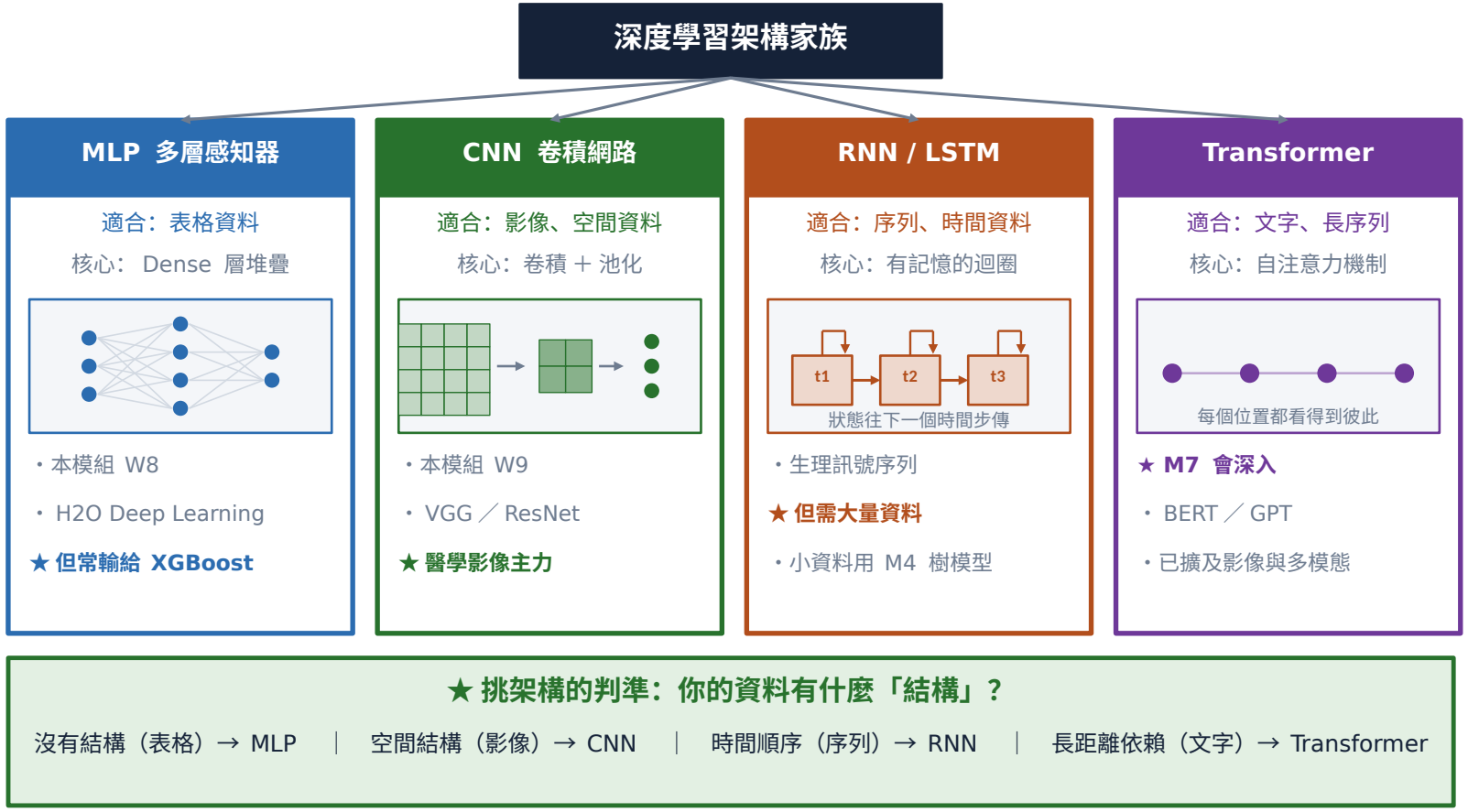
② 學習率要調小

- 微調時用比從零訓練「小一個數量級」的學習率（如 1e-4 → 1e-5）
- ★ 學習率太大會把預訓練好的權重破壞掉——這叫災難性遺忘
- 常見作法：解凍的層用小學習率、新加的分類層用正常學習率

③ 輸入格式必須對齊

- 預訓練模型有固定的輸入尺寸與正規化方式（如 224×224 、特定的均值標準差）
- ★ 醫學影像常是灰階與非標準尺寸，要先轉換
- 轉換方式錯了，模型表現會莫名其妙地差

深度學習的四大架構家族



另有自編碼器（Autoencoder）：不預測標籤，而是學會「重建自己」—— 用於降維與異常偵測（接 M3 、 M4 ）。

MLP 多層感知器

表格資料。本模組 W8 。★ 但常輸給 XGBoost 。

CNN 卷積網路

影像與空間資料。本模組 W9 。醫學影像主力。

RNN / LSTM 循環網路

序列與時間資料。★ 需大量資料，小資料仍建議用 M4 的樹模型 + 滯後特徵。

★ Transformer 注意力架構

文字與長序列。 M7 會深入。近年已擴及影像與多模態。

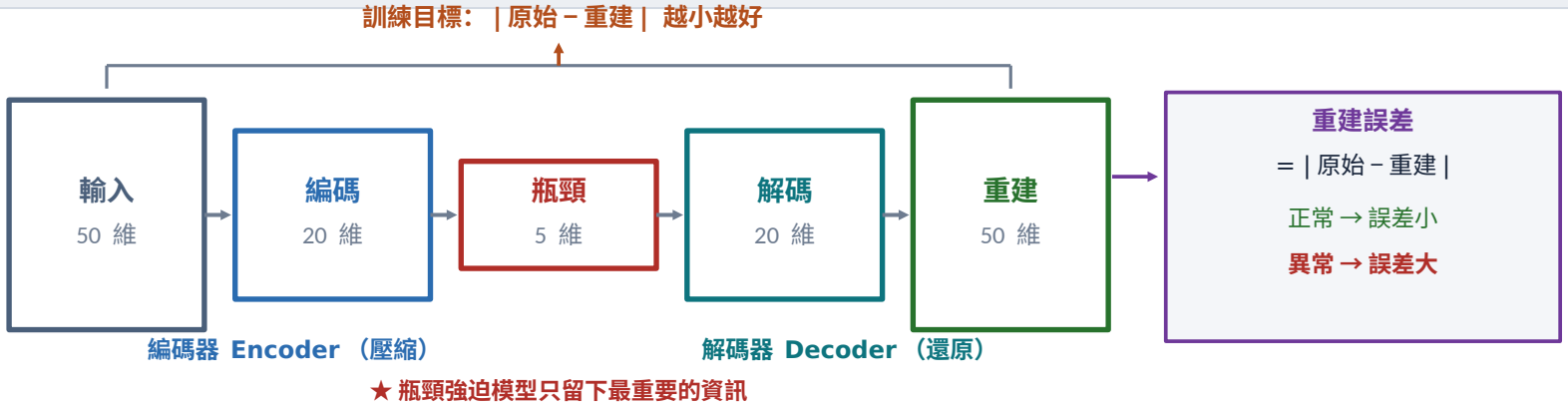
★ 原生圖形，可直接在 PowerPoint 編輯

四種架構的選用對照（含醫護場景）

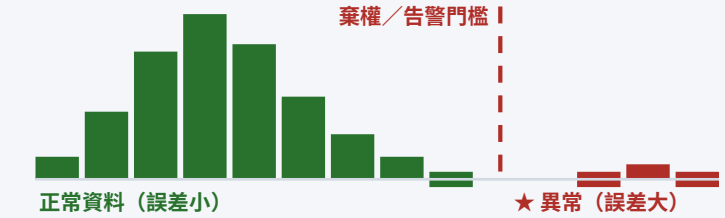
架構	資料的結構	醫護場景範例	替代方案（通常更務實）
MLP	無特殊結構（表格）	病歷欄位預測再入院	★ XGBoost （ M4 ）——多數情況表現更好且可解釋
CNN	空間結構（影像）	X 光、超音波、傷口照片	★ 遷移學習——幾乎不會從零訓練
RNN ／ LSTM	時間順序（序列）	連續生理監測訊號、病程序列	★ 滯後特徵 + 樹模型（ M4 ）——資料少時更穩
Transformer	長距離依賴（文字）	護理紀錄、出院摘要	★ 用現成預訓練模型（ M7 ），不要自己訓練

★ 注意最右欄：四種架構的「務實替代方案」全都不是自己訓練深度模型。這是本模組一再強調的立場。

自編碼器：學會「重建自己」



重建誤差的分布：門檻怎麼定



怎麼拿來偵測異常

① 只用「正常」資料訓練	讓它學會重建正常樣態
② 對新資料算重建誤差	｜原始 − 重建｜
③ 誤差超過門檻 → 異常	因為模型沒學過這種樣態

- M4 的 IQR、LOF、Isolation Forest 不需要先備妥正常資料；自編碼器需要一批「乾淨的正常資料」
- 優勢：能處理高維、非線性、影像等複雜資料；★ 也能同時當作降維工具（接 M3 的 PCA）
- 代價：需要算力、需要調參，且「重建誤差多大算異常」的門檻仍要自己定

★ 選用建議：先試 M4 的傳統方法。資料是影像／高維訊號、且傳統方法明顯不夠時，才動用自編碼器。

結構

輸入 → 壓縮到瓶頸 → 還原回原尺寸。
訓練目標是「輸出盡量等於輸入」。

★ 瓶頸是關鍵

強迫模型只留下最重要的資訊——
這與 M3 的 PCA 降維概念相通。

怎麼偵測異常

只用正常資料訓練 → 對新資料算重建誤差 → 誤差大＝模型沒學過這種樣態＝異常。

★ 與 M4 的關係

IQR、LOF、iForest 不需要一批乾淨的正常資料；
自編碼器需要一批乾淨的正常資料。先試傳統方法。

★ 原生圖形，可直接在 PowerPoint 編輯

實作 2 ｜ MNIST 手寫數字辨識與 Fine-tune

【AI Studio 範例】 Repository ▸ 8. Deep Learning/5 、 5.1 、 6 、 8

開啟檔案

```
Repository ▸ 8. Deep Learning/
  5 CNN Deep Learning for
    image classification -
    MNIST.rmp
  5.1 CNN Fine Tune model.rmp
  6 more layers - MNIST.rmp
  8 incremental batch
    training.rmp
  MNIST mini/ （訓練與測試影像）
```

預期結果

- ▸ 卷積核數量減少 → 速度變快但準確率下降
- ▸ ★ 更多層不一定更好， MNIST 這種簡單任務尤其明顯
- ▸ Fine-tune 的訓練時間遠短於從零訓練
- ▸ 增量訓練讓模型可以隨新資料持續更新

操作步驟

1. 先跑範例 5 ，記錄準確率與訓練時間
2. ★ 打開網路設定，對照 PART 5 的流程圖，找出每一層對應哪個運算
3. 把卷積核數量從 32 改成 8 ，觀察準確率與速度的變化
4. 跑範例 6 （更多層），比較是否真的更好
5. ★ 跑範例 5.1 Fine-tune ，觀察它凍結了哪些層
6. 跑範例 8 （增量訓練），理解模型可以「續訓」而不用重來
7. 把三種設定做成一張表：準確率、訓練時間、參數量

PART 08

★ 什麼時候不該用深度學習

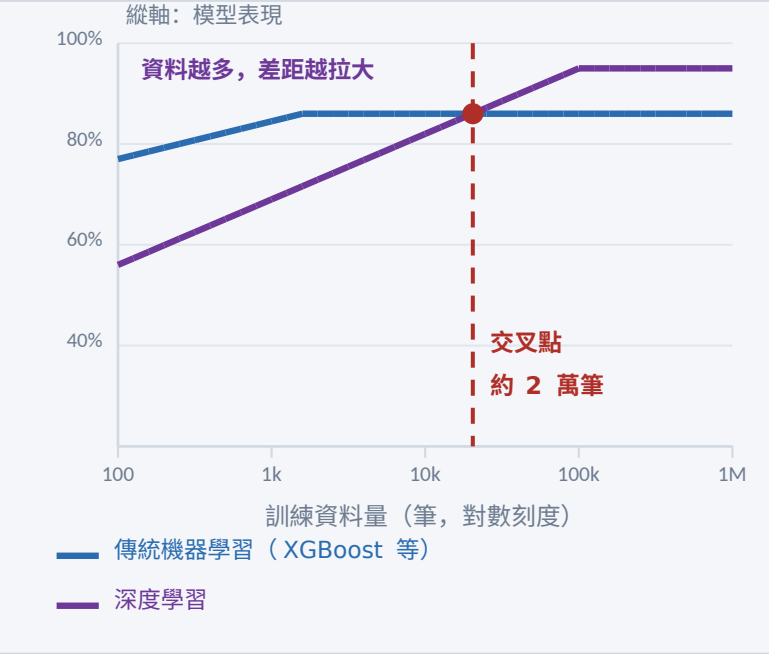
When NOT to Use Deep Learning

這是本模組最重要的一節。

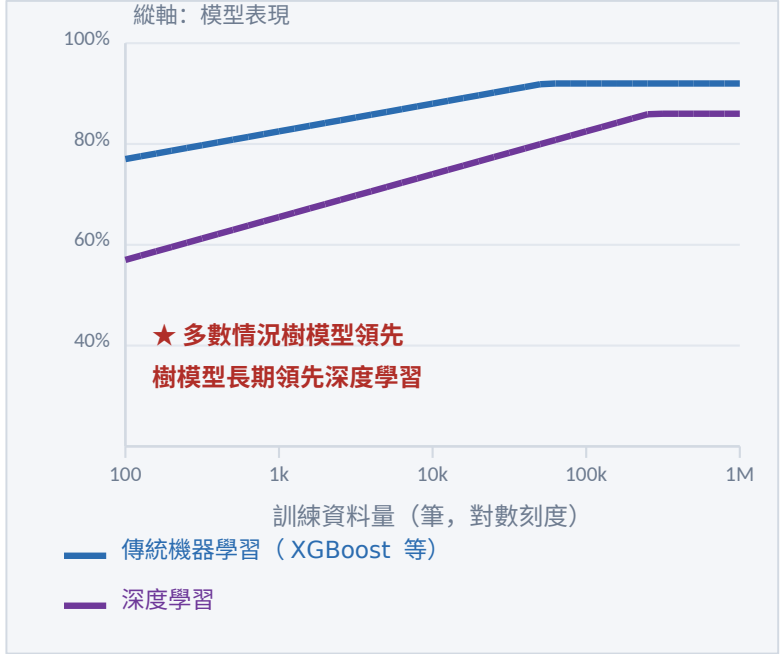
會用是基本功；知道什麼時候不用，才是專業判斷。

深度學習什麼時候才會贏：兩種資料型態

非結構化資料（影像、語音、長文本）



★ 表格資料（病歷欄位、檢驗值、行政資料）



★ 基準研究多支持此結論（Grinsztajn 2022）；但 2025 年 TabPFN 在萬筆以下小表格已可勝過梯度提升。

★ 對醫護領域的意義：你手上的資料絕大多數是表格資料 —— M4 的梯度提升樹往往才是對的工具。

非結構化資料（左）

影像、語音、長文本。資料量夠大時，深度學習會超越傳統方法，而且差距持續擴大。

★ 表格資料（右）

病歷欄位、檢驗值、行政資料。多數情況樹模型領先（Grinsztajn et al., 2022）。

基準研究怎麼說

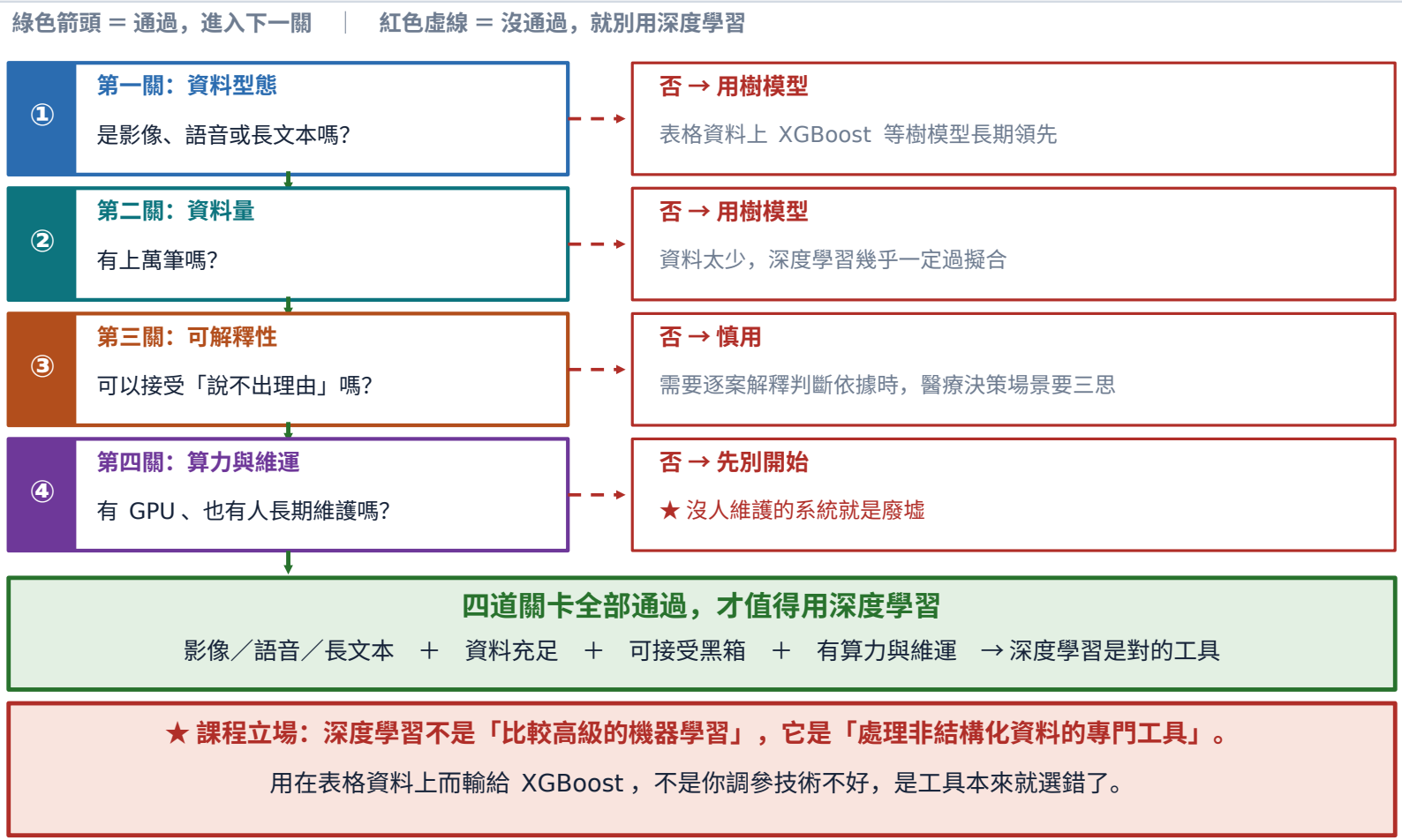
Grinsztajn et al.（2022）等研究：表格資料上樹模型多半領先；2025 年 TabPFN 在小表格開始挑戰此結論。

★ 對醫護領域的意義

你手上的資料絕大多數是表格資料。這意味著 M4 的梯度提升樹往往才是對的工具。

★ 原生圖形，可直接在 PowerPoint 編輯

什麼時候「不該」用深度學習：四道關卡



① 資料型態

不是影像／語音／長文本 → 用樹模型。

② 資料量

沒有上萬筆 → 從零訓練幾乎一定過擬合（遷移學習例外）。

③ 可解釋性

需要逐案解釋判斷依據 → 慎用。
醫療決策場景要三思。

④ 算力與維運

沒有 GPU 與維護能力 → 先別開始。
★ 沒人維護的系統就是廢墟。

★ 原生圖形，可直接在 PowerPoint 編輯

同一個問題，四種工具的實際比較

情境	資料型態	建議工具	理由
預測 30 天再入院	病歷欄位 8,000 筆（表格）	★ XGBoost （M4）	表格資料、量不大、需要解釋 → 深度學習沒有優勢
胸部 X 光異常篩檢	影像 3,000 張	★ 遷移學習（凍結卷積層）	影像任務、但資料少 → 不能從零訓練
護理紀錄分類	文字 5,000 筆	TF-IDF ＋ 邏輯斯迴歸（M6） 或微調小型語言模型	資料量中等；先試傳統方法建立基準線
急診人次預測	時間序列 3 年日資料	★ 樹模型 + 滯後特徵（M4）	資料量太小， LSTM 等序列模型會過擬合
語音轉文字	音訊	★ 現成預訓練模型（M7）	不要自己訓練——用 Whisper 等成熟模型

★ 注意：五個情境中，只有第二個真的需要你自己訓練深度學習模型，而且還是用遷移學習。這相當接近實務比例。

PART 09

醫學影像的特殊挑戰

Medical Imaging: Special Considerations

把 CNN 用在醫學影像上，會遇到教科書不會提的問題。

醫學影像深度學習的六個特殊挑戰

① 資料量永遠不夠

標註要專科醫師，成本極高

對策：遷移學習 ＋ 資料增強 ＋ 跨機構合作

② 類別極度不平衡

陽性個案往往不到 5%

對策：代價敏感（M4）＋看 Recall 與 PR 曲線

★ ③ 捷徑學習

模型學到的可能不是病灶

★ 曾有模型靠「片子上的醫院標記」判斷

④ 跨機構泛化差

換一台機器、換一家醫院就掉分

對策：外部驗證是必要的，不是加分項

⑤ 標註本身有歧異

兩位放射科醫師可能判讀不同

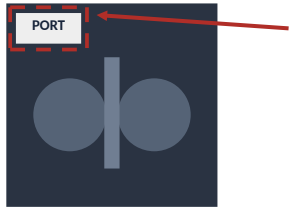
對策：多人標註 ＋ 算 Kappa

⑥ 法規與責任

醫材法規、臨床決策責任歸屬

★ 課堂專題請避開「診斷」類應用

★ 第③點最值得警惕：模型準確率很高，學到的卻是「錯的東西」



模型真正在看的地方

（攜帶式 X 光機的標記）

著名案例：某肺炎判讀模型在原醫院 AUC 0.93，換到外院明顯下降（Zech et al., 2018）。

事後發現它學的是「攜帶式 X 光機的標記」—— 因為重症病人才用攜帶式。

★ 準確率、 AUC 都很漂亮；只有做外部驗證（換一家醫院）才會露餡。

① 資料量永遠不夠

標註要專科醫師，成本極高。
對策：遷移學習 ＋ 資料增強 ＋ 跨機構合作。

② 類別極度不平衡

陽性個案往往不到 5%。
對策：代價敏感（M4）＋看 Recall 與 PR 曲線。

★ ③ 捷徑學習

模型學到的可能不是病灶——
曾有模型靠「片子上的醫院標記」判斷。

④⑤⑥

跨機構泛化差、標註本身有歧異、法規與責任歸屬。

★ 原生圖形，可直接在 PowerPoint 編輯

捷徑學習：最值得警惕的一種失敗

著名案例

- 某肺炎判讀模型在原醫院 AUC 0.93，換到外院明顯下降（Zech et al., 2018）
- ★ 事後發現：它學的是「攜帶式 X 光機的標記」——因為重症病人才用攜帶式
- 模型從未學會看肺部，它學會了看標記

為什麼難發現

- 準確率、 AUC 都很漂亮，所有指標都通過
- ★ 只有做「外部驗證」（換一家醫院的資料）才會露餡
- 或用可解釋性工具看模型在看哪裡（M9 會談 Grad-CAM）

怎麼防範

- ★ 外部驗證是必要條件，不是加分項——換機構、換機型都要測
- 用 Grad-CAM 等工具檢視模型的注意力落在哪裡（M9）
- 前處理時移除可能洩漏的資訊（標記、日期、機器型號）

★ 這一頁的觀念在 M9 （倫理與可解釋性）會再深入。捷徑學習不只發生在影像，表格資料一樣會。

實例：糖尿病視網膜病變 AI—— 從論文到診間

論文裡：跟專科醫師一樣好

- Gulshan et al., JAMA 2016：約 12.8 萬張眼底照訓練 CNN（Inception-v3），兩個驗證集敏感度 90.3% ／ 87.0%、特異度 98.1% ／ 98.5%
- 2018 年 IDx-DR 成為 FDA 第一個不需醫師判讀的自主式 AI：基層診所前瞻性試驗 900 人，敏感度 87.2%、特異度 90.7%
- 前瞻性、在診所實際拍照的數字比回溯資料集低——這就是 M2-2 講的外部驗證落差

到了診間：泰國 11 家診所的實地研究（Beede et al., CHI 2020）

- 護理師在診所拍的照片，超過五分之一被系統判定品質不足、無法判讀——光線不足是主因
- 影像要上傳雲端，網路慢就延誤；當場轉診讓病人擔心交通與費用，有護理師反而勸病人別參加
- 模型沒變，變的是拍照環境、網路與流程——準確率之外，還要問「在這裡能不能被用」

對台灣與你的專題

- 衛福部臨床 AI 取證驗證中心的首批實測，就包括眼底影像判讀糖尿病視網膜病變（見 M0）
- 跨院、跨相機的差異要靠資料增強（下一頁）與遷移學習（PART 06），但不能取代外部驗證
- 影像判讀屬醫材管制：課堂專題用公開資料練習，不做臨床診斷宣稱

資料增強：小資料的必要手段

手法	做什麼	醫學影像可不可以	注意
水平翻轉	左右鏡像	★ 要小心：心臟在左邊，翻轉後解剖位置就錯了	胸腔影像通常不做左右翻轉
旋轉	小角度旋轉（±10°）	✔ 通常可以	角度太大會偏離真實拍攝條件
裁切與縮放	隨機裁切一部分	✔ 可以，但別裁掉病灶	★ 病灶在邊緣時要小心
亮度／對比調整	模擬不同曝光	★ ✔ 很有用	模擬不同機器與參數的差異
加雜訊	加入隨機雜訊	✔ 可以	模擬影像品質變異
彈性形變	局部拉伸變形	✔ 常用於醫學影像	模擬器官的自然形變

★ 資料增強的原則：增強後的影像「在真實世界中應該有可能出現」。左右翻轉的胸腔 X 光在真實世界不會出現——那不是增強，是製造錯誤資料。

PART 10

自動化與收尾

AutoML & Wrap-up

讓工具幫你試參數，你把時間花在判斷上。

超參數最佳化：三種搜尋策略

① 網格搜尋 Grid Search

- 把每個參數的候選值排列組合，全部試一遍
- 優點：完整、可重現　缺點：組合數指數成長
- ★ 適合：參數少（2–3 個）且候選值明確時

② 隨機搜尋 Random Search

- 在參數空間中隨機取樣
- ★ 研究顯示：同樣的預算下，通常比網格搜尋更快找到好解
- 原因：多數參數其實不重要，網格搜尋浪費太多次在它們身上

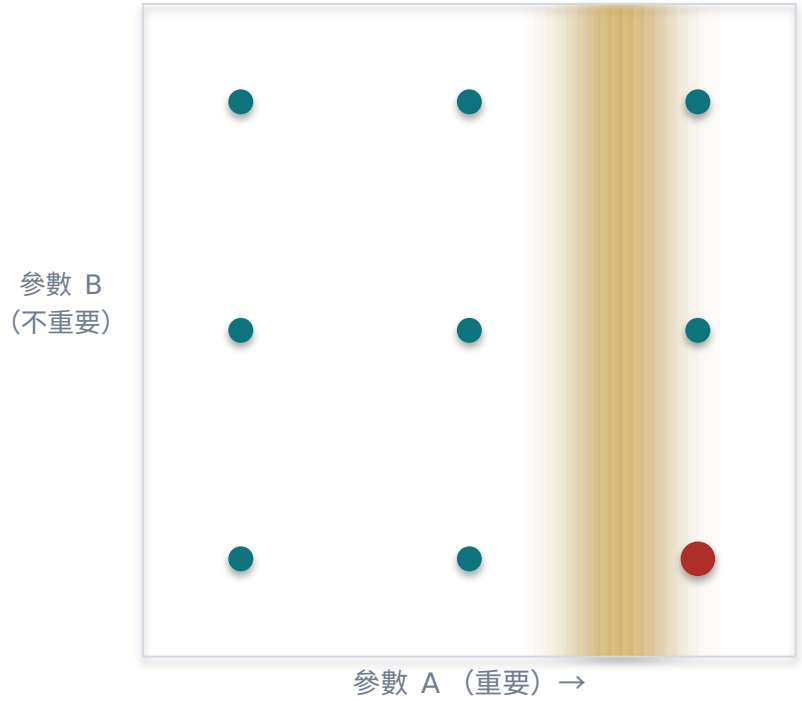
③ 貝氏最佳化 Bayesian Optimization

- 根據已試過的結果，推測下一個最有希望的參數組合
- ★ 試驗次數少時效率最高； AI Studio 內建 Grid 與 Evolutionary，貝氏需 Python （如 Optuna）
- 代價：較難平行化（因為要看前面的結果）

★ 實務順序：先手動試幾組建立感覺 → 再用隨機搜尋擴大範圍 → 最後用貝氏最佳化精修。不要一開始就跑大網格。

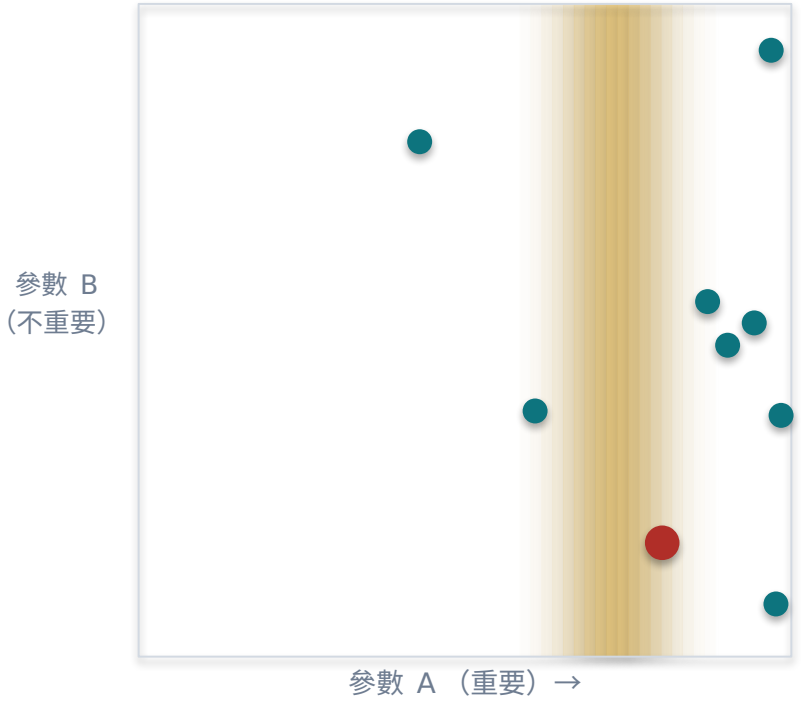
實例：網格搜尋 vs 隨機搜尋——同樣只試 9 次

網格搜尋（3 × 3） 找到的最佳分數 0.791



只試了 3 個不同的 A 值，另外 6 次浪費在不重要的 B 上

隨機搜尋（9 次） 找到的最佳分數 0.831



9 次就試了 9 個不同的 A 值，比較容易碰到好區域

橘色帶＝參數 A 的好區域（最佳在 0.73 附近）；參數 B 完全不影響分數。真實情況常是少數參數重要、多數不重要，所以同樣預算下隨機搜尋通常更快找到好解。實務順序：手動試幾組 → 隨機搜尋擴大範圍 → 貝氏最佳化精修。

實作 3 ｜自動模型選擇與超參數最佳化

【AI Studio 範例】 Export-Import/08 、 Repository ▸ 8. Deep Learning/7

開啟檔案

```
Export-Import/
  08-Automatic Model Selection
    and Optimization.rmp

Repository ▸ 8. Deep Learning/
  7 Telcon Deep Learning.rmp
  Deep Learning for regression
    - CLV Deep Learning.rmp
```

預期結果

- ▸ 網格搜尋的時間成本很快就變得無法接受
- ▸ ★ 演化式搜尋通常用更少次數找到相近或更好的解
- ▸ 在 Telco 這類表格資料上，梯度提升樹（GBT）很可能勝出
- ▸ 匯出的模型可以在沒有訓練資料的環境載入使用

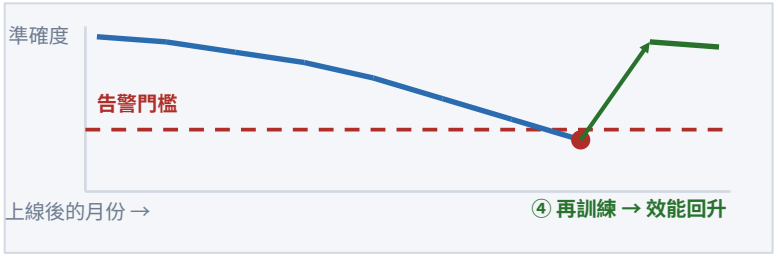
操作步驟

1. 先手動跑三組不同的網路設定，記錄結果（建立感覺）
2. ★ 用 Optimize Parameters (Grid)，設定 2 個參數各 3 個值 = 9 組
3. 觀察執行時間——注意組合數是相乘不是相加
4. 改用 Optimize Parameters (Evolutionary)，比較找到的最佳解
5. ★ 用同一份資料跑 M4 的 Gradient Boosted Trees，與最佳深度學習設定比較
6. 把模型匯出，再用 Apply Model 載入，確認可重複使用
7. 填寫下一頁的模型比較表

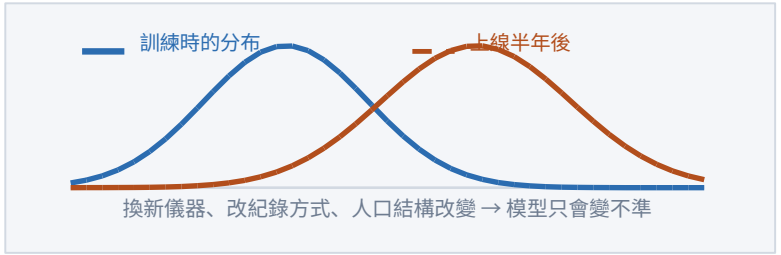
模型的一生：訓練只是開始

① 訓練	② 部署	③ 監測	④ 再訓練
一次性，但可能重跑很多次	把模型放進實際流程	上線後才是開始	模型會過期
• GPU 時數或雲端費用	• 模型匯出與版本管理	★ 資料漂移：輸入分布慢慢改變	• 新資料進來要不要重訓
• 資料標註成本（往往最貴）	• 推論的硬體需求	• 效能衰退偵測	• 重訓後要重新驗證
★ 實驗次數比你想的多 10 倍	• 與現有系統的介接	• 誰負責看、多久看一次	★ 沒有人力做＝系統會死

③ 上線後：模型會安靜地變差



為什麼會掉：輸入分布慢慢改變（資料漂移）



★ 多數學生專題只做到①。真正決定系統存活的是③與④ —— 而那需要的是人與流程，不是技術。

寫專題報告時把③④也寫進去（即使你沒做），會讓評審知道你懂得完整的生命週期。

① 訓練

★ 實驗次數比你想的多 10 倍。
資料標註成本往往是最貴的一項。

② 部署

模型匯出、版本管理、與現有系統介接。

★ ③ 監測

資料漂移：輸入分布會慢慢改變。
誰負責看、多久看一次？

★ ④ 再訓練

沒有人力做這件事＝系統會死。
多數學生專題只做到①。

★ 原生圖形，可直接在 PowerPoint 編輯

資料漂移：上線後最常見的失效原因

① 什麼是資料漂移

- 訓練時的資料分布，與上線後遇到的資料分布不同
- 醫療場域的常見來源：換了新的檢驗儀器、改了紀錄方式、人口結構改變、季節或疫情因素
- ★ 模型不會報錯，它會安靜地變得越來越不準

② 怎麼偵測

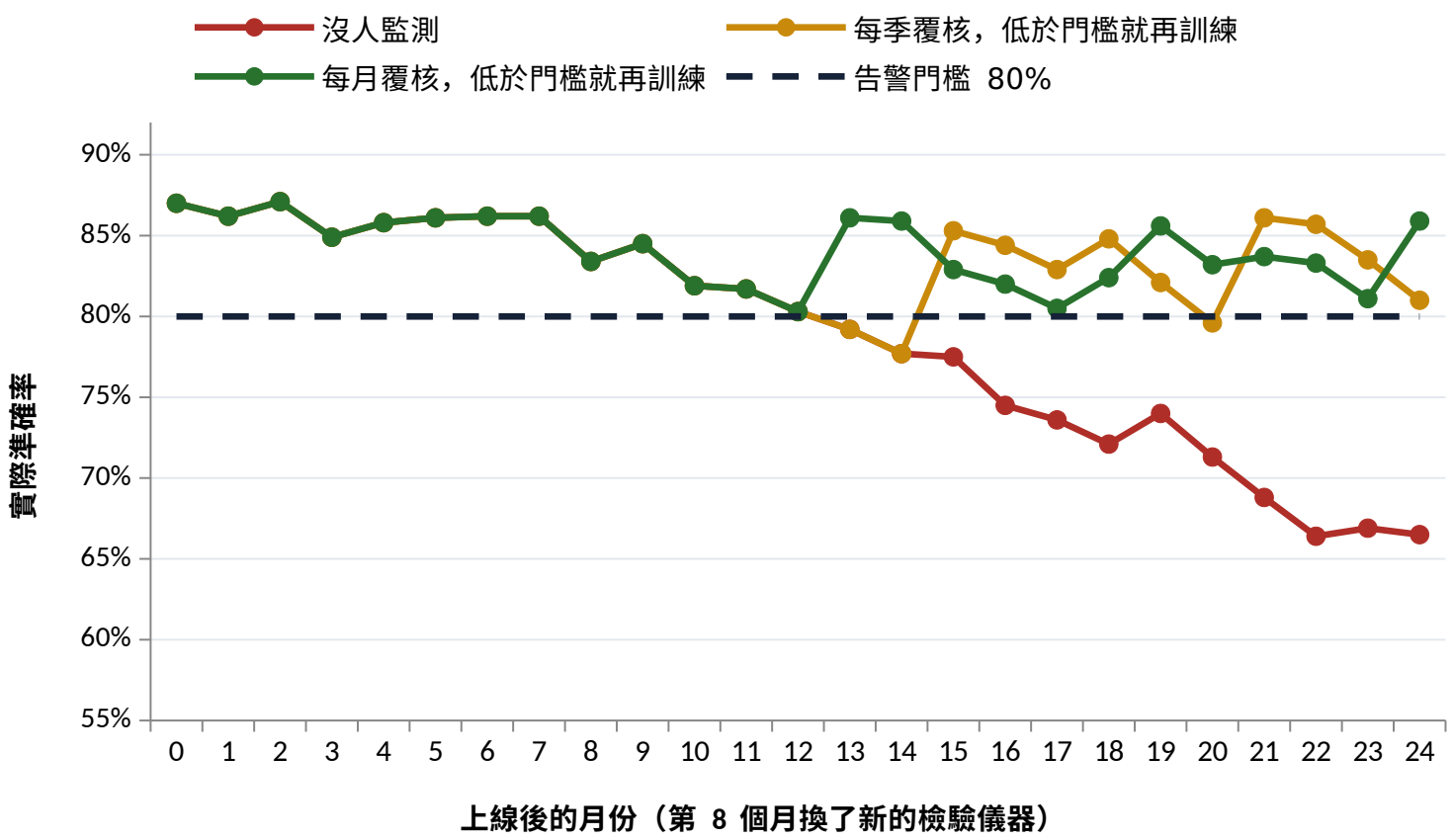
- 監測輸入特徵的分布（平均、標準差、缺值率）有沒有明顯改變
- 監測預測分布：預測為陽性的比例有沒有異常變化
- ★ 最直接：定期抽樣人工覆核，算實際準確率

③ 怎麼處理

- 設定門檻與告警：分布偏移超過多少就通知
- 定期重訓（如每季）；重訓後一定要重新驗證
- ★ 上線前就要把這件事寫進交付文件，指定負責的人

★ 這是「模型上線之後誰管」的問題。技術上不難，難在組織裡沒有人被指派做這件事。

實例：上線後，模型會安靜地變差



沒人監測

模型不會報錯，只會越來越不準： 24 個月裡有 12 個月低於門檻，最後掉到 66% 。

每季覆核

低於門檻的月數 3 個，再訓練 2 次。

每月覆核

低於門檻的月數 0 個，再訓練 3 次。技術不難，難在組織裡要有人被指派做這件事。

模擬：上線準確率 86% ，第 8 個月起每月漂移 1.2 個百分點。監測頻率越低，掉在門檻下的時間越長；重訓後一定要重新驗證。上線前就要把監測寫進交付文件、指定負責的人。

深度學習的十個常見錯誤（依發生頻率排序）

#	錯誤	後果	怎麼避免
1	忘記標準化特徵	訓練不收斂或極慢	★ 一律先做 Z-score
2	資料量太少卻用大網路	嚴重過擬合	小資料時參數量別超過樣本數
3	用測試集調參	分數虛高，上線就崩	切三份：訓練／驗證／測試
4	標準化參數用全部資料算	資料洩漏	★ 只用訓練集算，再套用
5	多標籤問題用 Softmax	機率被迫加總為 1	多標籤要用 Sigmoid
6	沒開早停	白白訓練到過擬合	一律開啟
7	只看準確率	類別不平衡時毫無意義	看 Recall、PR-AUC（M2-2）
8	學習率沒調就說模型不好	Loss 亂跳或不動	★ 先降學習率再說
9	沒有跟簡單模型比	不知道有沒有進步	★ 一定要有梯度提升樹基準線
10	時間序列資料隨機切分	嚴重洩漏	依時間切（M4）

★ 第 9 項最常被忽略：很多人花兩週調深度學習，最後才發現 XGBoost 跑 5 分鐘就贏了。永遠先建立基準線。

W8–W9 兩週的課堂節奏

時段	內容	形式	產出
W8 前半	PART 1–2：定位、神經網路運作原理	講述 + 手算示範	理解前向傳播
W8 後半	PART 3–4：Layers 設計、訓練與過擬合	講述 + 實作 1	Iris/Titanic 逐步加深的結果表
W8 課後	作業二發布	自主	選定資料集
W9 前半	PART 5–6：CNN、卷積手算、遷移學習	講述 + 實作 2	MNIST 結果表
W9 後半	★ PART 7–10：經典架構、何時不該用、醫療影像、自動化	★ 討論為主 + 實作 3	選型判斷的論述
W9 課後	作業二進行，W12 繳交	自主	模型比較表 + 論述

★ W9 後半刻意安排成討論課。這一節的內容不是技術，是判斷——講述效果有限，要讓學生自己說出理由。

隨堂檢核｜深度學習的判斷題

Q 1

你有 3,000 筆病歷欄位資料要預測再入院，同事建議用深度學習。你怎麼回應?

先問三件事：① 這是表格資料，多篇基準研究顯示 XGBoost 在表格上長期領先；② 3,000 筆對深度學習太少，幾乎一定過擬合；③ 再入院預測通常需要解釋依據。建議先用 XGBoost 建立基準線，若真的不夠再考慮——而且要能說出深度學習預期改善什麼。

Q 2

Loss 曲線顯示訓練持續下降、驗證從第 12 個 epoch 開始上升。你會做什麼?

這是典型過擬合。處理順序：① 先設早停在第 12 個 epoch （免費且立即有效）；② 檢查參數量是否超過樣本數，若是則減少層數或神經元；③ 才考慮加 Dropout 。不要一開始就加 Dropout—— 那治標不治本。

Q 3

你的 X 光判讀模型在院內測試準確率 94% ，可以導入了嗎?

不行。至少還要做：① 外部驗證（換一家醫院或換機型的資料）——捷徑學習就是這樣被抓出來的；② 用 Grad-CAM 檢視模型在看哪裡，請放射科醫師判斷是否合理；③ 確認類別不平衡下的 Recall 與 PR-AUC ，不是只看準確率；④ 釐清法規與責任歸屬。

Q 4

為什麼說「深度學習不是比較高級的機器學習」?

因為它是專門處理非結構化資料（影像、語音、長文本）的工具，不是通用升級。用在表格資料上輸給 XGBoost ，不是調參技術不好，是工具選錯了。選對工具是專業判斷，不是技術能力問題。

Take-Home Assignment ｜ M5 深度學習

★ 即為「作業二：深度學習分類器設計」 · W8 發布、W12 繳交 · 占學期成績 15%

作業二說明

【任務一】網路架構實驗 （40%）
資料：Pima Indians Diabetes（768 筆，課程指定；先處理偽裝成 0 的遺失值，見 M1）
1. 建立基準線：Logistic Regression（M2-2）與 Gradient Boosted Trees（M4）各跑一次，記錄準確率與 AUC
2. 深度學習設定至少五組，例如：
   [10] / [50,50] / [200,200] / [200,200,200,200] / [50,50]+Dropout0.3
3. 每組記錄：訓練分數、驗證分數、兩者差距、訓練時間
4. ⋆ 附上至少兩張 Loss 曲線截圖（一張健康、一張過擬合）

【任務二】CNN 影像分類 （35%）
資料：MNIST mini（或自選公開影像資料集，不得使用真實醫療影像）
1. 跑通基本 CNN，記錄準確率
2. 改變「卷積核數量」與「層數」各做兩組變化
3. ⋆ 做一組 Fine-tune（範例 5.1），比較訓練時間與準確率
4. 把三組結果做成表

【任務三】★ 判斷與論述 （25%） —— 這一題分數最重
1. 綜合任務一與任務二，回答：
・在你的表格資料上，深度學習有贏過 Logistic 與梯度提升樹嗎？
・如果沒有，你認為原因是什麼？（從資料型態與資料量分析）
2. 假設你要在服務單位導入其中一個模型，你會選哪一個？請說明：
・技術理由（表現、成本、可解釋性）
・非技術理由（誰維護、出錯怎麼辦、如何說服同仁）
3. 字數 400 字以上

【繳交】PDF 報告（含流程截圖、Loss 曲線、結果表）＋ .rmp 流程檔，壓縮為一個檔案。檔名：M5_學號_姓名
【提醒】★ 不得使用任何真實醫療影像或病歷資料。

作業二任務一紀錄表：基準線＋五組網路

設定	訓練準確率	驗證準確率	差距（> 0.05 標過擬合）	AUC	訓練時間（秒）
Logistic Regression （基準）					
Gradient Boosted Trees （基準）					
hidden [10]					
hidden [50, 50]					
hidden [200, 200]					
hidden [200, 200, 200, 200]					
hidden [50, 50] ＋ Dropout 0.3					

先跑兩個基準線（淡黃底），再逐步加深網路。差距＝訓練 − 驗證；附至少兩張 Loss 曲線截圖（一張健康、一張過擬合）。

最常見的結論：深度學習最好的設定也贏不了 Gradient Boosted Trees—— 在表格資料上這很正常，寫進報告就是好的發現。網站版可直接填寫並下載 CSV 。

本模組重點回顧

1. 深度學習與傳統 ML 的最核心差別：★ 誰決定特徵
2. 神經網路 = 加權求和 + 非線性轉換，重複幾百萬次。沒有魔法
3. 沒有激活函數， 100 層等於 1 層 —— 非線性是深度學習的存在前提
4. ★ 訓練時 80% 的時間在看 Loss 曲線；學會判讀四種樣態最重要
5. 過擬合處理順序：早停 → 減少容量 → 才加 Dropout 與正則化
6. ★ 小型表格資料：參數量別超過訓練樣本數 —— 否則是背答案

M5 名詞速查表　深度學習

（1 / 6）

中文	English	一句話說明
深度學習	Deep Learning	以多層神經網路自動學習表徵的方法
表徵學習	Representation Learning	由模型自行學出有用特徵，而非人工設計
神經元	Neuron / Unit	執行加權求和再通過激活函數的基本運算單元
權重與偏差	Weight & Bias	神經元的可學習參數
前向傳播	Forward Propagation	由輸入逐層計算到輸出的過程
反向傳播	Backpropagation	以連鎖律把誤差往回傳，計算各權重梯度
梯度下降	Gradient Descent	沿梯度反方向更新權重以降低損失
學習率	Learning Rate	每次更新的步伐大小；訓練不穩時第一個該檢查的參數
優化器	Optimizer	梯度下降的改良演算法，如 Adam 、 RMSProp

M5 名詞速查表　深度學習

（2 / 6）

中文	English	一句話說明
動量	Momentum	累積過去更新方向，加速收斂並衝過小坑洞
Adam	Adaptive Moment Estimation	結合動量與自適應學習率；多數情況的預設選擇
激活函數	Activation Function	引入非線性；沒有它多層網路等於單層
ReLU	Rectified Linear Unit	max(0,x)；隱藏層的預設激活函數
Sigmoid	Sigmoid	輸出 0–1；二分類輸出層使用
Softmax	Softmax	多分類輸出層；各類機率加總為 1
梯度消失	Vanishing Gradient	梯度往回傳時逐漸趨近 0，前層學不動
全連接層	Dense / Fully-connected Layer	每個神經元連接前一層所有神經元
Dropout	Dropout	訓練時隨機關閉部分神經元以防過擬合

M5 名詞速查表　深度學習

（3 / 6）

中文	English	一句話說明
批次正規化	Batch Normalization	將每層輸出重新標準化，加速收斂
Epoch	Epoch	整個訓練集被完整掃過一次
批次	Batch / Mini-batch	一次送入網路的樣本數
迭代	Iteration	一次權重更新；等於一次前向加一次反向
損失函數	Loss Function	衡量預測與真實的差距；訓練要最小化的目標
交叉熵	Cross-Entropy	分類任務的標準損失函數
早停	Early Stopping	驗證分數不再進步即停止訓練
L1 ／ L2 正則化	L1 / L2 Regularization	對權重大小施加懲罰以抑制過擬合
卷積	Convolution	以卷積核在輸入上滑動做局部加權求和

M5 名詞速查表　深度學習

（4 / 6）

中文	English	一句話說明
卷積核	Kernel / Filter	可學習的小矩陣，用來偵測特定特徵
特徵圖	Feature Map	卷積運算的輸出
步幅	Stride	卷積核滑動的間隔
補零	Padding	在輸入邊緣補零以維持輸出尺寸
感受野	Receptive Field	輸出的一個位置對應原始輸入的範圍
池化	Pooling	以取最大或平均的方式縮小特徵圖
平移不變性	Translation Invariance	物體位置改變時輸出仍相近
展平	Flatten	把多維特徵圖攤平為一維向量
卷積神經網路	Convolutional Neural Network, CNN	以卷積與池化為核心的影像處理架構

M5 名詞速查表　深度學習

（5 / 6）

中文	English	一句話說明
殘差連接	Residual Connection / Skip Connection	讓輸入跳過數層直接相加，解決深層梯度消失
ResNet	Residual Network	採用殘差連接的深層架構；現代視覺骨幹
遷移學習	Transfer Learning	以預訓練模型為起點，套用到新任務
預訓練模型	Pre-trained Model	已在大型資料集訓練完成的模型
凍結	Freezing	固定某些層的權重不參與訓練
微調	Fine-tuning	解凍部分或全部層，以小學習率繼續訓練
災難性遺忘	Catastrophic Forgetting	微調學習率過大導致預訓練知識被破壞
資料增強	Data Augmentation	以旋轉、裁切等變換擴充訓練資料
捷徑學習	Shortcut Learning	模型學到與任務無關但有相關性的線索

M5 名詞速查表　深度學習

（6 / 6）

中文	English	一句話說明
外部驗證	External Validation	以其他機構或設備的資料測試模型泛化能力
超參數最佳化	Hyperparameter Optimization	系統化搜尋最佳參數組合
網格搜尋	Grid Search	窮舉所有參數組合
隨機搜尋	Random Search	在參數空間隨機取樣；相同預算下常優於網格
貝氏最佳化	Bayesian Optimization	依據已試結果推測下一組最有希望的參數

課後複習：本單元的互動學習站

掃描 QR code 或輸入網址；手機、平板、電腦都能用

掃我開啟 M5 互動版

手機相機對準即可開啟

網址

nchsieh.github.io/assets/courses/001/interactive/m5/

22 節，順序對應講義各 PART

看完一個 PART 就做對應的那一節，不必一次做完

改參數、立刻看結果

圖表可以拖拉調整，看懂「為什麼」，而不只是背結論

隨堂練習＋即時回饋

答錯會說明原因；作答進度只存在你自己的瀏覽器

全部單元入口： nchsieh.github.io/assets/courses/001/interactive/ · 網站內容為教學示例與模擬資料，不需登入